

Guidelines for Collaboration

Zhiyang Ong*

May 14, 2026

Copyright © 2026 by Zhiyang Ong

This work is licensed under a [Creative Commons “Attribution-NonCommercial-NoDerivatives 4.0 International”](#) license.



Abstract

This is a set of guidelines for development/design processes, methodologies, and practices, as well as conduct while collaborating on open source projects, spanning engineering, computer science, data science, and network science. It also includes guidelines for creating a shared BibTeX database.

Contents

Revision History	2
1 Guidelines for Conduct	3
2 Guidelines for Creating a Shared BibTeX Database	4
2.1 Recommended Fields for BibTeX Entries	8
2.2 Template Information for the Annote BibTeX Field	12
2.3 Template Information for the Howpublished BibTeX Field	17
2.4 Additional Recommendations for Managing a Shared BibTeX Database	17
3 Coding Standard	18
4 Exception Safety	20
5 Suggested Software Architecture	20
6 Adoption of Best Practices	21
6.1 Practice of Automated Regression Testing	22
6.2 Cloud-based Data Science and Machine Learning	23
7 Intellectual Property Protection, via Licenses	23

*Email correspondence to: ongz@acm.org

8 Additional Guidelines	23
8.1 Suggested Guidelines for Benchmarking	24
8.2 Suggested Prefixes for Labels of Parts of a $\text{\LaTeX}$ Document	25
Acknowledgments	25
To-Do List	26
References	26

Revision History

Revision history:

1. Version 1, October 2, 2014. Initial version of the guideline (for another project).
2. Version 1.1, December 23, 2014. Version ported for this boilerplate code project.
3. Version 2, October 20, 2015. Added guidelines for *Doxygen*-supported, *Javadoc*-based coding standard. This coding standard is also known as coding style, coding style guide, coding guideline, coding scheme, code convention, code documentation guideline, programming guideline, or programming style.
4. Version 2.1, October 21, 2015. Finished guidelines for *Doxygen*-supported, *Javadoc*-based coding standard for *C++*.
5. Version 2.2, June 4, 2016. Finished section for additional guidelines: to include documentation using *Markdown*, and tools for software development, integrated circuit and cyber-physical system design, and documentation.
6. Version 3, November 3, 2016. Added guidelines for: documenting *GNU Octave* and MATLAB code, in order to facilitate documentation generation using *Texinfo* [373–375, 422]; sharing of source code, design files, sets of benchmarks, data sets, and documentation on online repositories [124, 150]; and added section on exception safety.
7. Version 3.1, November 4, 2016. Fixed references for indent style conventions.
8. Version 3.2, December 20, 2016. Update guidelines for conduct.
9. Version 3.3, February 3, 2017. Update information about usage of *GitHub*’s services.
10. Version 3.4, March 11, 2017. Update information on naming convention.
11. Version 3.5, October 9, 2017. Update guidelines on commenting/writing code.
12. Version 3.6, December 24, 2017. Fix grammatical error in a sentence.
13. Version 3.7, January 25, 2018. Added suggestions for software architecture of my computer programs.
14. Version 3.8, January 31, 2018. Added information about coding style guideline for different computer languages, and also about online repositories that facilitate research reproducibility, replicability, and repeatability.
15. Version 3.9, February 1, 2018. Added information about developing software in a *Pythonic* style.
16. Version 4.0, June 8, 2018. Added information about specifying (co-)authors’ full name, and research reproducibility, and other best practices from software development, and embedded/cyber-physical system and integrated circuit design.
17. Version 4.1, September 19, 2018. Updated ACM Code of Ethics and Professional Conduct; added The Joint ACM/IEEE-CS Software Engineering Code of Ethics and Professional Practice; and updated guidelines on exception handling.

18. Version 4.2, September 21, 2018. Added acknowledgements, shout outs, for people who helped me with automated regression testing. And, refactored document.
19. Version 4.3, September 22-25, 2018. Added references on agile SoC design, and hardware/VL-SI/RTL/HDL refactoring.
20. Version 4.4, December 29-30, 2018. Added template information for the **Annote** and **Howpublished** **BIBTEX** fields to help me identify particular information about the document/publication.
21. Version 4.5, February 11, 2019. Extend recommendations and suggestions to projects in data science and (applied) machine learning.
22. Version 4.6, February 14, 2019. Add references for *Git* and its substitute, *Mercurial SCM*, for distributed version/revision control (or software configuration management). Also, add “Acknowledgments” and “Revision History” sections to the “Table of Contents”.
23. Version 4.7, March 13, 2019. Add comment on how to include mathematical expressions in *Markdown* documents. In addition, fix minor errors.
24. Version 4.8, January 22, 2020. Fix **BIBTEX** key error (renamed for a publication), and added references regarding design by contract (or programming by contract, or contract programming) and Hoare logic.
25. Version 4.9, March 11, 2021. Fix minor errors in guidelines for creating shared **BIBTEX** databases.
26. Version 5.0, June 20, 2021. Refactor §2.2 to include information about publications in the **Annote** field of **BIBTEX** entries.
27. Version 5.1, October 31, 2021. Add a To-Do list.
28. Version 5.2, April 25, 2022. Updated document with clarifications, in the abstract and conduct guideline.
29. Version 5.3, June 4, 2022. Updated document with information about converting text into all uppercase or lowercase, which is useful for writing dissertations and theses.
30. Version 5.4, June 4, 2022. Updated §6 on best/good practices for software development, integrated circuit design, and cyber-physical system design projects for more clarity.
31. Version 5.5, June 5, 2022 - April 11, 2023. Minor updates to add necessary clarifications for collaboration, including information about crafting (and sharing) **BIBTEX** databases.
32. Version 5.6, April 13-15, 2023. Added information on how to fix bugs in **BIBTEX** databases.
33. Version 5.7, April 16-17, 2023. Added values to exhibit during collaboration.
34. Version 6.0, May 20, 2024. Updated information on using software development and data science practices to improve I.C. design.
35. Version 6.1, November 17, 2025. Changed license to Creative Commons license for the automatically generated PDF document from the corresponding **L^AT_EX** source file.
36. Version 6.2, November 18, 2025. Update revision history.
37. Version 6.3, February 15, 2026. Add information to deal with retracted/withdrawn publications.
38. Version 6.4, February 22, 2026. Modify information to deal with multiple versions of the title.
39. Version 6.5, March 27, 2026. Add/modify information on how to indicate the references for **BIBTEX** fields in the **Annote** **BIBTEX** field.
40. Version 6.6, March 27, 2026. Standardize mentions/references of **BIBTEX** keys as: BibTeX-KEY (singular) and BibTeX-KEYs (plural).

1 Guidelines for Conduct

Collaborators of open source software and/or hardware projects that we are involved in should follow the *Code of Conduct* of the *Institute of Electrical and Electronics Engineers* (IEEE) [97–99] and the

ACM Code of Ethics and Professional Practice of the Association for Computing Machinery (ACM) [5, 21, 51–53, 161–163, 433], including the “The Joint ACM/IEEE-CS Software Engineering Code of Ethics and Professional Practice” [164, 165]. Also, actions of discrimination are not acceptable [100]; we should intentionally commit to inclusive diversity, equity, justice, and accessibility [334] as well as honesty [49, 65, 70, 102, 145, 172, 221, 251, 254, 274, 285, 305, 321, 328, 329, 349, 350, 383] and accountability [6, 46, 63, 74, 75, 80, 91, 108, 114, 126, 128, 138, 141, 154, 156, 273, 301, 302, 328, 333, 344, 365, 378, 394, 411]. In addition, we should embrace the values and principles of agile software development [36, 67, 304]. An additional guideline is “Dave Packard’s 11 simple rules” [179].

While collaborating, we should practice ethical and healthy conflict resolution (Cite this!!!), and engage in scalable compassion (Cite this!!!) by working on projects for social, economic, and environmental good (Cite this!!!). We should also exercise intellectual humility and servant leadership (Cite this!!!).

In addition, when there is a dispute about which technology, algorithm, design paradigm/style/-pattern, process, or methodology to use, follow or adhere to the “Code Wins Arguments” [218, 441] guideline. Also, when considerable effort has been invested in an automated regression testing/verification infrastructure, do not be afraid to “move fast and break things” [115, 127].

Lastly, we should adopt a mission-focused and value-based approach to participate in meetings and discussions for the project(s). We should be intellectually curious (CITE THIS!!!) and flexible/liberal enough to consider and explore viable alternate approaches to do things and solve problems [35, 36]. Where disputes occur, a data-driven, fact-based approach based on the “Code Wins Arguments” guideline should be used to resolve conflicts.

2 Guidelines for Creating a Shared BIB_TE_X Database

Guidelines for creating BIB_TE_X entries and the BIB_TE_X database, which is used for writing the paper, are given as follows:

1. Each BIB_TE_X key should be unique:
 - (a) Check if your desired BIB_TE_X key already exists in the BIB_TE_X database:
 - i. If it does, and the BIB_TE_X entry for the publication already exists in the BIB_TE_X database, do not add it to the BIB_TE_X database.
 - ii. Else if the desired BIB_TE_X key exists in the database, but the BIB_TE_X entry for the publication does not exist in the BIB_TE_X database, repeat the process with an updated BIB_TE_X key (add the letter “a”, select the next letter in the English alphabet, or pick the next number), and add information about the publication to the BIB_TE_X database.
 - iii. Else if the desired BIB_TE_X key and associated publication do not exist in the database, add it to the BIB_TE_X database.
 - (b) Use the following format for creating BIB_TE_X keys: [first] author’s last name, appended by the year of publication. E.g., my first conference paper would have the BIB_TE_X key Ong2004. If the year of publication is not known, use an approximate year, with XY for the last 2 digits in the year (e.g., 20XY). Alternatively, if you cannot determine if it was published this

millennium or the previous millennium (or much earlier), use UNKNOWN for the “year”. For example, use KleinbergUNKNOWN (preferred for unknown millennium), or Smith20XY (for unknown year in the 21st century).

- i. For (last) names that end with a number, append the year to it anyway. We can still process the last four characters/digits of the year to determine the year of publication. I assume that there are no publications before 0 C.E. (common era).
- (c) Remove duplicate entries in the BIB_TE_X database. **WARNING! Before doing this, perform a union operation on the fields of the BIB_TE_X entries. For example, if a BIB_TE_X entry has information that the other BIB_TE_X entry does not have, or vice versa, merge the information to one BIB_TE_X entry.:**
 - i. **Rationale: Duplicate BIB_TE_X entries will cause problems in typesetting.**
- (d) Regarding hash collision of BIB_TE_X keys, such as multiple instances of Gratz2014, distinguish them by appending a letter to them. E.g., use Gratz2014, Gratz2014a, Gratz2014b, Gratz2014c, and so on. If we run out of letters, append it with “a” followed by a number. The use of the letter “a” separates the year from the instance of BIB_TE_X key. That is, Gratz2014a2 tells me that it is the 29th instance of Gratz2014, as opposed to Gratz201429.
- (e) If possible, restrict the characters of each BIB_TE_X key to be alphanumeric. The year is always numeric, and is appended to the (first) author’s last name.
 - i. If the (first) author’s last name has characters with diacritical marks, accents, or diacritics, remove/trim the characters used to typeset the diacritical marks (or accents) from the (first) author’s last name, and append the year of publication to it. E.g., *Söménzi* (year 2000) becomes *Somenzi2000*.
 - ii. If the (first) author’s last name has characters that are not letters in English, anglicize those characters. Unless specified by the co-authors, we should avoid using the transliteration for a given non-English language, since such transliteration may not be standardized (for non-commonly spoken/used languages). Also, supporting letters from other languages is a tedious task. Hence, we can use the anglicized version of their last names instead.
- (f) For reports, manuals, booklets, and other publications that list the author as the name of the organization publishing it, specify the author field and BIB_TE_X key as follows:
 - i. Author = {[Name-of-Organization] staff}},
 - A. **Append the word “staff” to the name of the organization, and use this as the author’s name.** Ensure that this name would typeset as it appears as one token by surrounding/encapsulating this name with round brackets (or parentheses). The name of the organization in this example is given within square brackets.
 - ii. Its BIB_TE_X key, for the publication year of 20XY, is: NameOfOrganizationStaff20XY.
 - A. For names of organizations with multiple words, concatenate the words while capitalizing the first letter of each word (even for articles and conjunctions, such as “of”, “the”, and “and”). This makes the BIB_TE_X key easier to read.
 - B. If the name of an organization is a capitalized acronym, such as CIAO, concatenate its name with the word “staff” without capitalizing the first letter “s” in “staff”. This also makes the BIB_TE_X key easier to read. An example for a publication year of 20XY is: CIAOstaff20XY.
 - C. Similarly, if the name of an organization ends with a capital letter, such as TrentoSMT, concatenate its name with the word “staff” without capitalizing the first letter “s” in “staff”. This also makes the BIB_TE_X key easier to read. An example for a publication year of 20XY is: TrentoSMTstaff20XY.

number of BIB_TE_X keys into multiple L^AT_EX `\cite` commands [417, in §2 Citations: §2.4 No cite].

- ii. Use the `typeset_to_pdf.py` *Python* script [287] to obtain the set of BIB_TE_X keys from a BIB_TE_X database, and include them in citations using multiple L^AT_EX `\cite` commands, in a standalone L^AT_EX file that can be included in a simple L^AT_EX template for typesetting; see https://github.com/eda-ricercatore/bibtex-analytics/blob/master/typeset_to_pdf.py:

A. Alternatively, an *OCaml* program/script for this task is: [INSERT-URL-HERE!!!].

- (b) Typeset the L^AT_EX document:
 - i. Typesetting the L^AT_EX document using the L^AT_EX `\nocite{*}` command should run faster than using a L^AT_EX `\cite` command or multiple L^AT_EX `\cite` commands.
- (c) Check that the font and style of the reference list is correct.
- (d) If there are errors, correct the errors as appropriate.
- (e) Finally, the BIB_TE_X database should be correct.

10. Information that I would include when citing common sources of information, such as *Wikipedia*, using the Harvard Referencing Style [258, 259, 397, 398, 428, 429]:

- (a) Wikipedia contributors, “TITLE_OF_THE_ARTICLE,” in {*\it* Wikipedia, The Free Encyclopedia: CATEGORY}, Wikimedia Foundation, San Francisco, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (b) Wikibooks contributors, “CHAPTER_NAME,” in {*\it* TITLE_OF_THE_BOOK}, Wikibooks: Open books for an open world, Wikimedia Foundation, San Francisco, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (c) Wikibooks contributors, “SECTION,” in {*\it* CHAPTER} of {*\it* TITLE OF THE BOOK}, Wikibooks: Open books for an open world, Wikimedia Foundation, San Francisco, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (d) Wikibooks contributors, “TITLE_OF_THE_BOOK,” Wikibooks: Open books for an open world, Wikimedia Foundation, San Francisco, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (e) Wikiquote contributors, “TITLE,” Wikiquote, Wikimedia Foundation, San Francisco, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (f) Wiktionary contributors, “TITLE,” Wiktionary, Wikimedia Foundation, San Francisco, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (g) Dictionary.com, “WORD,” IAC, Oakland, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (h) AUTHOR, “TITLE,” in {*\it* The New York Times: The Opinion Pages: Op-Ed Contributor}, The New York Times Company, New York, NY, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (i) AUTHOR, “QUESTION,” in {*\it* CATEGORY}, Quora, Inc., Mountain View, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (j) AUTHOR, Answer to “QUESTION,” in {*\it* CATEGORY: QUESTION}, Quora, Inc., Mountain View, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.
- (k) AUTHOR, “TITLE OF POST,” in {*\it* BLOG TITLE}, Quora, Inc., Mountain View, CA, MONTH DATE, YEAR. Available online at: `\url{URL}`; last accessed on August 26, 2014.

- (l) AUTHOR, “TITLE,” Stack Exchange Inc., New York, NY, MONTH DAY, YEAR. Available online from {\it Stack Exchange Inc.: Stack Overflow: Questions} at: \url{URL}; March 16, 2016 was the last accessed date.
 - (m) AUTHOR, “TITLE OF REPOSITORY,” GitHub, Inc., San Francisco, CA, MONTH DAY, YEAR. Available online from {\it {GitHub: GitHub user’s (full) name or USERNAME (or NAME OF ORGANIZATION)}: at: \url{URL}; March 16, 2016 was the last accessed date.
 - (n) AUTHOR, “TITLE OF PAPER,” Cornell University, Ithaca, NY, MONTH DAY, YEAR. Available online (as Version XYZ) from {\it {arXiv: FIELD(s)}: at: \url{URL}; March 16, 2016 was the last accessed date.
 - (o) When BIB_TE_X entries are created for the aforementioned sources of information, populate the appropriate fields so that each information in the aforementioned sources are included in the BIB_TE_X entries.
 - (p) **For other organizations, communities, and groups, use the term “contributors” instead of “members” to indicate that only a subset of them contributed to a publication, unless otherwise specified.** Else, use “members”.
11. Refer to the file “bibtex-template.txt” for templates for selected BIB_TE_X entry types. The more information that you can put in, the easier you can protect yourself from accusations of plagiarism and to make it easier for people (including yourself) to find the reference again. This is especially true for Web-based references/resources.
 12. When the names of authors and editors are unknown, list them as “Anonymous contributors”:
 - (a) An exception is made for a list of anonymous professors; use the following instead: [Name-of-Organization/University] professors.

2.1 Recommended Fields for BIB_TE_X Entries

The recommended fields for BIB_TE_X entries are:

1. booklet:
 - (a) Address
 - (b) Author or Editor
 - (c) Edition
 - (d) Howpublished
 - (e) Keywords
 - (f) Month
 - (g) Pages
 - (h) Publisher
 - (i) Series
 - (j) Title
 - (k) URL
 - (l) Volume
 - (m) Year
2. techreport:
 - (a) Address
 - (b) Author or Editor
 - (c) DOI
 - (d) Howpublished
 - (e) Institution

- (f) Keywords
- (g) Month
- (h) Number
- (i) Title
- (j) URL
- (k) Year

3. proceedings:

- (a) Address
- (b) DOI
- (c) Editor
- (d) Keywords
- (e) Month
- (f) Organization
- (g) Publisher
- (h) Series
- (i) Title
- (j) Volume
- (k) Year

4. manual:

- (a) Address
- (b) Author or Editor
- (c) Howpublished
- (d) Keywords
- (e) Month
- (f) Number
- (g) Organization
- (h) Title
- (i) URL
- (j) Year

5. inbook:

- (a) Address
- (b) Author or Editor
- (c) Booktitle
- (d) Chapter (optional)
- (e) DOI
- (f) Edition
- (g) Howpublished
- (h) Keywords
- (i) Month
- (j) Number
- (k) Pages
- (l) Publisher
- (m) Series
- (n) Title
- (o) URL

(p) Volume

(q) Year

6. incollection:

(a) Address

(b) Author or Editor

(c) Booktitle

(d) Chapter

(e) DOI

(f) Edition

(g) Howpublished

(h) Keywords

(i) Month

(j) Number

(k) Pages

(l) Publisher

(m) Series

(n) Title

(o) URL

(p) Volume

(q) Year

7. inproceedings:

(a) Address

(b) Author

(c) Booktitle

(d) DOI

(e) Keywords

(f) Month

(g) Number

(h) Organization

(i) Pages

(j) Publisher

(k) Series

(l) Title

(m) Volume

(n) Year

8. article:

(a) Address

(b) Author

(c) DOI

(d) Journal

(e) Keywords

(f) Month

(g) Number

(h) Pages

(i) Publisher

- (j) Title
- (k) Volume
- (l) Year

9. phdthesis (or mastersthesis):

- (a) Address
- (b) Author
- (c) DOI (there are multiple research universities that assign DOIs to Ph.D. dissertations, and possibly Masters theses)
- (d) Howpublished
- (e) Keywords
- (f) Month
- (g) Number:
 - i. Some B_IB_TE_X bibliography styles don't indicate the number, or show the number but hide some other important information.
 - ii. Hence, if the number is important include it in the Annote field.
- (h) School
- (i) Title
- (j) URL
- (k) Year
- (l) Annote

10. misc:

- (a) Address
- (b) Author
- (c) Howpublished
- (d) Keywords
- (e) Month
- (f) Publisher or School
- (g) Title
- (h) URL
- (i) Year

11. book:

- (a) Address
- (b) Author
- (c) DOI
- (d) Edition
- (e) Keywords
- (f) Month
- (g) Pages
- (h) Publisher
- (i) Series
- (j) Title
- (k) Volume
- (l) Year

For additional notes and annotations for publications, I can use the BIB_TE_X fields **Annote** or **Note** to include that information in the BIB_TE_X entry. Please kindly note that information from the BIB_TE_X field **Note** may end up in the reference list (or list of references), while information from the BIB_TE_X field **Annote** would not.

In addition, to cite the specific page numbers of interest, use the BIB_TE_X field **Pages**.

2.2 Template Information for the Annote BIB_TE_X Field

This is a list of template information for the **Annote** BIB_TE_X field to help me identify particular information about the document/publication:

1. Alternate names of authors: blah, blah blah, blah blah blah, ...
 - (a) Deprecated:
 - i. Authors' names (alternate): BLAH-1 and BLAH-2.
 - ii. Full names of authors: BLAH-1 and BLAH-2.
 - iii. Alternate names of [Author XYZ]: BLAH-1 and BLAH-2.
 - iv. Actual name of author: BLAH.
 - v. Actual name of co-authors: BLAH and BLAH.
 - vi. No authors are associated with this document, but the authors associated with the BibTeX-KEY document are also associated with this document; they are published as a set. Hence, instead of using anonymous contributors, I use their names instead.
 - (b) Previously indicated the staff or members of the following organizations as co-authors, instead of individuals: ORG-ABC, ORG-DEF, and ORG-HIG.:
 - i. For the author field, we should use names of individuals or their staff/community members (in cases when individual contributors are unknown). E.g., "NVIDIA staff" as the value for the author field, only if the specific employees at NVIDIA who contributed to the publication are unknown.
 - (c) Previously indicated set of co-authors: Blah-1, Blah-2, Blah-3, and Blah-4.
 - (d) Alternate order of co-authors, since the order between the front cover and title page (and copyright page) is different): Blah-1, Blah-2, Blah-3, and Blah-4.
 - (e) For publications that many contributors, select the major contributors to have their names listed and include the following to complete the list of co-authors: "and other contributors", or "and other [PROJECT-NAME] contributors".
 - i. This allows us to indicate the subset of members of the organization or community that contributed to the publication, data set, or project, instead of the more inclusive "members" term.
2. Addressing the problem of ghostwriters:
 - (a) Ghostwriters for the book are: BLAH.
3. Addressing the problem of namesakes:
 - (a) This is a namesake of: NAME-OF-NAMESAKE.
 - (b) This is a namesake of: NAME-OF-NAMESAKE-1, NAME-OF-NAMESAKE-2, NAME-OF-NAMESAKE-3, and NAME-OF-NAMESAKE-4.
 - i. The (co-)author has the following alternate names: *NAME-OF-NAMESAKE-1*, *NAME-OF-NAMESAKE-2*, *NAME-OF-NAMESAKE-3*, and *NAME-OF-NAMESAKE-4*.
4. Editors: BLAH ... BLAH ... BLAH:
 - (a) When citing from this entry, shift the names of the authors from the author field to the editor field.

- i. Since certain BIB_TE_X styles require a non-empty author field, I have placed the information of editors in the authors field.
 - ii. Swap them back when using them for publications, by changing the **Author** field to the **Editor** field.
 - (b) Editors of the book: BLAH-1 and BLAH-2.
 - (c) Editors of conference proceedings: BLAH-1, BLAH-2, and BLAH-3.
 - (d) Series editors: BLAH-1 and BLAH-2.
5. Publication issues:
- (a) Repeated/duplicate BIB_TE_X entries:
 - i. Repeat entry. See `\cite{BibTeX-KEY}`.
 - ii. Merged with the entry for BibTeX-KEY, since they refer to the same publication.
 - (b) Dates of publication:
 - i. Originally published in: 20XY. See BibTeX-KEY.
 - ii. Also, published in: 20XY. See BibTeX-KEY.
 - iii. Conference proceedings published in: 20XY.:
 - A. Dates of the conference are: MONTH DAY1-DAY2, YEAR.
 - B. Dates of the workshop are: MONTH DAY1-DAY2, YEAR.
 - iv. Reprinted in 20XY by BLAH. See BibTeX-KEY.
 - v. Copyright renewed in 20XY.
 - vi. Years of publication for previous editions: First edition in 19XY-1, Second edition in 200X-1, Third edition in 201X-1, and Fourth edition in 202X-1.
 - vii. Previously indicated dates are probably incorrect: DATE-1, DATE-2, and DATE-3.
 - viii. Date of publication is probably incorrect: DATE. Repeat entry. See `\cite{BibTeX-KEYs}`.
 - ix. 20X1, 20X2, and 20X3 are the alternate years of publication, cited by: SOURCES.
 - x. 20XY is the alternate year of publication, cited by: SOURCE.
 - xi. Possible year of publication: 20XY.
 - xii. Year of publication may be incorrect: 20XY.
 - xiii. Date of publication may be incorrect: MONTH, DAY, YEAR.
 - (c) Addresses of publisher(s):
 - i. Address of publisher, rather than conference location is: BLAH.
 - ii. Address of conference location, rather than publisher is: BLAH.
 - iii. Address is officially stated as: BLAH.
 - iv. Alternate address: BLAH.
 - v. Address of conference venue is previous indicated as: BLAH.:
 - A. Address of conference venue is previous indicated as: Muenchen, Germany. It is a German name (or rather, München, Germany) of Munich, Germany.
 - vi. Deprecated templates:
 - A. Address is officially stated in: BLAH.
 - (d) Alternate **names** of publisher:
 - i. Alternate publisher name: BLAH.
 - ii. Unknown publisher; see the following for an alternate BibTeX entry with an associated publisher: BibTeX-KEY.
 - iii. Unknown publisher; see the following for an alternate BibTeX entry without an associated publisher: BibTeX-KEY.
 - (e) Organizations associated with the publication:
 - i. Organizations that are recognized as involved in this publication: BLAH-1 and BLAH-2.
 - ii. Alternate names of organizations that are recognized as involved in this publication: BLAH-1 and BLAH-2.
 - iii. Deprecated templates:

- A. BLAH-1 and BLAH-2 are recognized as organizations involved in this report.
- (f) **Other publishers** of the publication:
- i. Previously indicated publisher is: BLAH.
 - ii. Additional publisher: BLAH.
 - iii. Distributed by: BLAH.
- (g) Alternate set of publication **titles** and **series** (and alternate **volume number**):
- i. Also, published as BLAH (old series title and old volume number).
 - ii. Alternate series title: BLAH.
 - iii. Regarding translations, and translated titles in a particular language, see the set of items for **Translation-based templates**.
 - iv. This title was used in the reprint in 20XY: BLAH-BLAH-BLAH.
- (h) Alternate titles of the publication:
- i. Alternate title: *Another title for the document/publication*.
 - ii. Alternate booktitle: *Another title for the conference proceedings*.
 - iii. Alternate subtitle: *BLAH*.
 - iv. Subsubtitle: *BLAH*.
 - v. Alternate subsubtitle: *BLAH*.
 - vi. Title of previous edition/printing: *Another title for the document/publication*.
 - vii. Differences from the online version with the print version:
 - A. Differences from the online version with the print version. Title of the print version on the cover of the magazine: How My Generation Broke America. Title of the online version: My Generation was Supposed to Level America’s Playing Field. Instead, We Rigged It for Ourselves.
 - B. Differences from the online version with the print version. Title of the print version: “How My Generation Broke America” (on the cover of the magazine). Title of the online version: “My Generation was Supposed to Level America’s Playing Field. Instead, We Rigged It for Ourselves” (online article). (deprecated)
 - viii. Differences from the front cover version with the title page version (in the front matter):
 - A. Differences from the front cover version with the title page version in the front matter. Title of the front cover version: “Beautiful Souls: The Courage and Conscience of Ordinary People in Extraordinary Times”. Title of the title page version in the front matter: “Beautiful Souls: Saying No, Breaking Ranks, and Heeding the Voice of Conscience in Dark Times”.
 - B. Differences from the front cover version with the title page version (in the front matter). Title of the front cover version: “Beautiful Souls: The Courage and Conscience of Ordinary People in Extraordinary Times” (on the front cover version). Title of the title page version (in the front matter): “Beautiful Souls: Saying No, Breaking Ranks, and Heeding the Voice of Conscience in Dark Times” (title page version). (deprecated)
 - ix. Actual title, not used in the **Title** field due to typesetting errors: *Actual title for the document/publication*.
- (i) Alternate series of publication:
- i. Possible series: BLAH.
 - ii. Series was previously known as: BLAH.
- (j) Alternate editions of publication:
- i. Reprinted, paperback edition.
 - ii. Previous editions are published by BLAH in: 1987, 1991, 1999, and 2009.
- (k) DOI problems:

- i. Warning: DOI does not link back to the Web page URL.
 - ii. Digital Object Identifier (DOI) redirects to the Web page for the BLAH-X edition of the book, while the PDF preview document on that Web page is for the BLAH-Y edition of the book.
- (l) URLs:
- i. Alternate URL: BLAH.
 - ii. Available online as `{\it The \LaTeX\ Project}` at: `\url{}`.
 - iii. Scans of the magazines are available at: URL.
 - iv. PDF copy of a draft of the publication is available at: URL.
 - v. PDF copies of the book can be downloaded on this Web page.
- (m) Subsets of publications:
- i. For the following BIB_TE_X entry types:
 - A. `incollection` (with titles for chapters, sections, subsections, and subsubsections), for the `book`, `booklet`, `techreport`, `phdthesis`, and `mastersthesis`, and `manual` BIB_TE_X entry types.
 - B. `inbook` (without titles for chapters, sections, subsections, and subsubsections), for the `book`, `booklet`, `techreport`, `phdthesis`, and `mastersthesis`, and `manual` BIB_TE_X entry types.
 - C. `inproceedings`, for the `proceedings` BIB_TE_X entry type.
 - D. `article`, for the `book` and `booklet` BIB_TE_X entry types.
 - ii. Part of the book/report: BLAH.
 - iii. This article is part of the journal/magazine issue: BibTeX-KEY.
- (n) Abstracts of publications listed as separate publications:
- i. Abstract is available from BLAH at: BLAH-BLAH-BLAH.
- (o) Multiple fields of publication:
- i. Originally published by BLAH, ADDRESS, in: YEAR.
 - ii. Originally published by BLAH in: YEAR.
 - iii. Originally published by BLAH, ADDRESS.
 - iv. Also, published by BLAH, ADDRESS, in YEAR.
 - v. Also, published by BLAH, ADDRESS.
 - vi. Deprecated templates:
 - A. Originally published by BLAH in BLAH-BLAH.
 - B. Also, published by BLAH in BLAH-BLAH.
- (p) Prepublication information:
- i. BibTeX entry completed with very limited prepublication information. This needs to be updated.
- (q) Retracted/Withdrawn publications:
- i. Retracted/withdrawn publication.
 - ii. Include the following keyphrases in the `Keywords` BIB_TE_X field to indicate that the publication has been retracted/withdrawn, so that searching for such publications in the BIB_TE_X database can be facilitated:
 - A. books - retracted
 - B. retracted books
6. **Translation**-based templates:
- (a) Translated by: BLAH.
 - (b) BibTeX key of English translation: BLAH.
 - (c) BibTeX key of original German version: BLAH.
 - (d) Translated English title: BLAH.
 - (e) Original French title: BLAH.

- (f) Originally published in German in the year: 20XY.
- (g) Original French series: BLAH.
- (h) Deprecated templates:
 - i. English translation of title: BLAH.
- 7. **Associated files for the publication:**
 - (a) Associated files are formerly named: BLAH.file_extension, ...
 - (b) Original filename of PDF copy of the publication: BLAH.pdf.
 - (c) Refer to BibTeX-KEY for a PDF copy of this publication.
 - (d) Refer to BibTeX-KEY for a PDF copy of this publication. Only the front matter is available in the PDF copy.
 - (e) PDF copy of this publication is not available.
- 8. Tracking updates to BIB_TE_X keys:
 - (a) Former BibTeX key is: Former BibTeX key.
 - (b) Former BibTeX keys are: Former BibTeX key #1 and Former BibTeX key #2 ...
 - (c) Keep BibTeX key as: BIPMJCGMWG2VIMcontributors2012. This enables the acronym to be differentiated from the term "contributors".
- 9. Similar publications:
 - (a) Similar dissertations: [List of BIB_TE_X keys].
 - (b) Similar publications: [List of BIB_TE_X keys].
 - (c) Differences between multiple publications, which are similar:
 - i. Differences between multiple publications are listed as follows. PUBLICATION1 addresses XYZ, but PUBLICATION2 and PUBLICATION3 do not.
- 10. Questionable correctness of BIB_TE_X entries:
 - (a) Questionable correctness of BibTeX entry. Information is probably obtained from secondary sources of information, such as publications that cite this publication.
- 11. Templates for the email receipt of a copy of the publication, such as a PDF copy:
 - (a) Received a copy of this *report/document/publication/thesis/dissertation* by email.
 - (b) Received a copy of this report by email.
- 12. Information on how to cite the publication, if it is located on a different URL (or Web page) than the URL included in the URL field of the BIB_TE_X entry, especially for the misc BIB_TE_X type.
 - (a) Information of how to cite this publication is provided as follows. Available online from {\it Home Page: Path to the Web page providing information on how to cite the publication} at: \url{URL}; MONTH DAY, YEAR was the last accessed date.
- 13. Common alternate names for the publication: BLAH.:
 - (a) E.g., "Dragon book" [426] refers to [11, 12].
 - (b) E.g., "CLRS" [427] refers to [83–85].
- 14. Book derived from Ph.D. dissertation: [BibTeX-KEY of Ph.D. dissertation].
- 15. Book derived from Master's thesis: [BibTeX-KEY of Master's thesis].
- 16. Undergraduate Honors/Senior thesis.
- 17. PDF copy of this publication is not available.
- 18. DOI of this publication is not available.
- 19. References for information used in BIB_TE_X fields:
 - (a) Information about the [BIB_TE_X field] is found from the following publication: [BibTeX-KEY].
 - i. E.g., "Information about the Author field is found from the following publication: ClimateChangeCommittee2026a."
- 20. Online repository of LaTeX sources for the book: [BibTeX-KEY].

2.3 Template Information for the Howpublished BIB_TE_X Field

For the Howpublished BIB_TE_X field, the suggested template information is:

- Available online at: \url{}; self-published; MONTH DAY, YEAR was the last accessed date.
- Available online from *main Web page: ABC section: XYZ subsection* as Version X.Y.Z at: \url{}; self-published; MONTH DAY, YEAR was the last accessed date.
- Available online from *main Web page: ABC section: XYZ subsection* in Italian at: \url{}; self-published; MONTH DAY, YEAR was the last accessed date.
- Available online from *main Web page: ABC section: XYZ subsection* in Italian and Spanish at: \url{}; self-published; MONTH DAY, YEAR was the last accessed date.

When the date is not known so that it can be provided in the “MONTH DAY, YEAR” format, use the following phrase instead of “MONTH DAY, YEAR was the last accessed date”.

- the last accessed date is unknown

If the information for the BIB_TE_X entry was obtained on multiple dates, include the following information in the Howpublished BIB_TE_X field:

1. BLAH-1 and BLAH-2 were the last accessed dates

2.4 Additional Recommendations for Managing a Shared BIB_TE_X Database

Use `crossref` for BIB_TE_X entries using the BIB_TE_X entry type `inproceedings` to share information about the conference proceedings without having to copy and paste fields that are commonly shared by articles/papers in conference proceedings [208, §12.2.3, pp. 234]. It might not work with BIB_TE_X entry types `inbook` and `incollection` for parts, subsections, sections, and chapters of books; this is not mentioned in BIB_TE_X specifications/references, and `crossref` is probably not supported for these BIB_TE_X entry types. Hence, I am currently not using `crossref`, unless I have to cite multiple (more than 15, > 15, or even five, > 5) conference papers in a given conference proceedings.

When copying (and pasting) text from a document or a Web page, non-ASCII hidden characters may be accidentally copied from the source (document or Web page) to the BIB_TE_X database. Consequently, this can cause command-line utilities for UNIX-like operating systems, such as `grep`, to fail to recognize the BIB_TE_X database as a text file. While some text editors or integrated development environments (IDEs) have features that highlight or indicate such non-ASCII hidden characters, they are not effective for finding/detecting these characters in large text files (BIB_TE_X databases in this case) without knowing what these characters are. If these characters are known, the search feature of these text editors or IDEs can find and delete these non-ASCII hidden characters.

Some alternatives are to use the L^AT_EX `\nocite{*}` command, or develop and use a script to concatenate all the BIB_TE_X keys (of a BIB_TE_X database) in a `\cite{}` L^AT_EX command. These options would enable the automatic generation of a list of the references in a particular BIB_TE_X style of your choice. This results in the L^AT_EX and BIB_TE_X interpreters parsing and processing your L^AT_EX and BIB_TE_X sources. If non-ASCII hidden characters exist in your L^AT_EX and BIB_TE_X source files, this process should abruptly pause your L^AT_EX and BIB_TE_X interpretation processes to warn you of the syntax errors due to these characters.

In addition, note that corruption in memory subsystems or storage devices (such as a solid-state drives) and data transfer between computers can result in corrupting text files to produce non-ASCII hidden characters in these files.

3 Coding Standard

This is a guideline for Doxygen-supported [404], Javadoc-based [239] coding standard that shall be used for software development projects, integrated circuit (IC) design projects, and cyber-physical system design projects. The term “coding standard” is used interchangeably/synonymously with coding style, coding style guide, coding guideline, coding scheme, code convention, code documentation guideline, programming guideline, or programming style. Our coding style/standard shall be self-documenting. The documentation generator that shall be supported is: Doxygen. This is because Doxygen can automatically generate software documentation for multiple computer languages, from programming languages to hardware description languages (HDLs). Since we are using Doxygen for generating documentation, we can use L^AT_EX to provide richer markup.

Support for hardware construction languages (HCLs) tends to be uncommon/rare, since their user base is still (fairly) small.

Document the known bugs for each function/method.

Our indent style would be the *1TBS* variant of the *K&R* style, which is an abbreviation of “*The One True Brace Style*”. It is also equivalent to the *Kernel Normal Form style* (or *BSD KNF style*) [419].

Classes, functions/methods, constants, macros, and static and instance variables shall be named using complete words or well-known abbreviations that are concatenated with an underscore in *C++*; this is a deviation from the *Hungarian notation* that uses an upper case letter to distinguish words/abbreviations in the name (i.e., the *Start case style of writing*; see letter case). That is, the naming convention followed is using multiple-word identifiers, via delimiter-separated words rather than letter-case separated words (e.g., *Hungarian notation*) [423].

For *C++* programs, the following tags shall be used in the comments:

1. @author *Author's_Name*: indicate the author (*Author's_Name*) of the file/function
 - (a) @modified by NAME, DATE in “Month Day, Year” format.
2. @version *X.Y*: indicate the version (*X.Y*) of the file
3. @section *SECTION_NAME*: indicate the section (*SECTION_NAME*) of the file, which can be: *LICENSE* or *DESCRIPTION*
4. @param *x*: indicate the parameter (*x*) of the constructor or function
 - (a) By default, use “@param None” to indicate that the function/method has no parameters.
5. @exception *Exception_Name*, or @throws *Exception_Name*: an exception that a function/method can throw
6. @return *Return_Statement*: indicate the return (type and) action of the function
7. @see *reference*: a link to another element in the documentation; e.g., @see *Class_Name*, or @see *Class_Name#member_function_name*
8. @since *X.Y: Month-Day-Year*: This functionality has been added since version *X.Y* (and on the date *Month-Day-Year*)
9. @deprecated *description*: Describe an outdated function/method, and indicate when the function/method has deprecated
10. “@link ... URL... @endlink” is used to include hyperlinks in the generated documentation for Doxygen
11. #### IMPORTANT NOTES: Notes that are critical for helping the reader understanding assumptions and decisions made while developing the software

12. @todo(<message>, <version>) (or ##### TO BE COMPLETED): Task to be finished at a later time. If it is busywork (or, busy work), indicate that it is busywork.
13. ##### TO BE FIXED: Task to be fixed at a later time, including:
 - (a) bugs to be debugged
 - (b) errors/faults to be fixed
 - (c) software/hardware/system architecture or source code to be refactored
 - (d) completion of feature implementation
14. @migration(<message>, <version>): Code is being migrated to another function/method, or class.
15. See <http://www.stack.nl/~dimitri/doxygen/commands.html> for more information of tags that are recognized by Doxygen.
16. @pre (or @precondition): Precondition(s) of the function.
17. @assert (or @assertion): Assertion(s) of the function.
18. @post (or @postcondition): Postcondition(s) of the function.

The order of tags in different sections of the *C++* code is given as follows:

1. Headers/Interfaces and Classes: @version, @author, @since, @link, @todo, @deprecated, @migration, and @see
2. Constructors: @param, @throws, @since, @link, @todo, @deprecated, @migration, and @see. For collaborators modifying or extending my code, they should include the @version and @author tags before the @param tag(s).
3. Functions/Methods: @param, @pre, @assert, @post, @return, @throws, @since, @link, @todo, @deprecated, @migration, and @see. For collaborators modifying or extending my code, they should include the @version and @author tags before the @param tag(s).
4. Variables can use the @see tags.
5. The @deprecated tag can be used for headers/interfaces, classes, constructors, functions/methods, and variables.

Additional coding style guidelines can be found in [31, 64, 87, 181, 269, 319, 362, 392].

For a suggested coding style for *Python* and *Ruby* scripts, see [295, 405] and [242], respectively. Regarding coding style guidelines for embedded *C*, see [30, 220]. In addition, there exists coding style guidelines for *Java* [66, 239, 288–291, 369] and *LabVIEW* [45, 81]. Coding style guidelines for *Verilog* can be found at: [39, 40]. Likewise, the coding style guide for *SystemVerilog* can be found at [268]. For other coding style guidelines, see [61, 119, 125, 187, 202, 207, 224, 253, 357, 360, 366, 403, 410, 432]. *Google* style guides [160] has provided documentation about best practices [297] for coding standards and the philosophy [296] of Google’s coding standards.

While well-documented source code is desired, natural language programming [420] is usually infeasible due to the choices of programming/computer languages used. Also, while literate programming [205, 206, 253, 279, 293, 357, 384] is encouraged, we are currently not following it due to the tedious process of developing software using literate programming. Hence, a short development time for well-commented, functionally correct, and efficient source code is prioritized over code written according to the literate programming approach.

4 Exception Safety

When developing software using programming/scripting languages that enable exceptions or errors to be thrown and caught, adopt "a set of contractual guidelines" [418] to support exception/error management. This "set of contractual guidelines" is based on exception safety guarantees in *C++* [3, 4, 418] [416, Subsection §4.4 on "Writing exception safe code"].

The levels of exception/error safety listed in descending order of safety guarantees are [3, 4, 416, 418]:

1. no throw guarantee, or failure transparency: "Best level of exception safety."
2. strong exception safety, commit/rollback semantics, or no-change guarantee
3. basic exception safety
4. minimal exception safety, no-leak guarantee
5. no exception safety: "No guarantees are made. (Worst level of exception safety)"

These aforementioned levels of exception/error safety can be partially handled. Also, the use of guards is strongly recommended for making the software and library (or, circuit or system) exception safe.

These guidelines about exceptions help software developers know what to do about fatal exceptions, boneheaded exceptions, vexing exceptions (due to unfortunate design decisions). Vexing exceptions and boneheaded exceptions, to a lesser extent, are preventable exceptions [238]. Hence, we should develop software that avoids triggering preventable exceptions.

Please judiciously consider what to do with the semipredicate problem [421].

5 Suggested Software Architecture

At the software system level, the software architecture can be described by the following modules/components:

1. parser(s):
 - (a) For input benchmarks
2. utilities:
 - (a) output generator(s)
 - (b) flag/switch -based printing information to standard output/error:
 - i. Print statements only when debugging mode is on.
 - ii. Else, squelch print/trace statements to speed up computation/performance.
3. solvers:
 - (a) ODE solver(s) for ordinary differential equations (ODEs):
 - i. ODE solver(s) for nonlinear ODEs.
 - (b) PDE solver(s) for partial differential equations (PDEs):
 - i. PDE solver(s) for nonlinear PDEs.
 - (c) satisfiability modulo theories (SMT) solver(s)
 - (d) boolean/proposition satisfiability (SAT) solver(s)
 - (e) maximum satisfiability modulo theories (Max-SMT) solver(s)
 - (f) maximum satisfiability (Max-SAT) solver(s)

- (g) pseudo-boolean optimization (PBO) solver(s)
 - (h) quadratic unconstrained binary optimization (QUBO) solver(s)
 - (i) weighted boolean optimization (WBO) solver(s)
 - (j) framework for algorithmic portfolio optimization
4. data structures:
- (a) directed graphs:
 - i. directed acyclic graphs (DAGs)
 - ii. binary decision diagrams (BDDs)
 - iii. AND-inverter graphs (AIGs)
 - (b) undirected graphs:
 - i. heaps
 - ii. trees
 - (c) maps, dictionaries, and hash tables
5. graphical user interface (GUI), if required.

Lastly, suggestions are not available for digital and mixed-signal integrated circuits (ICs) and very large-scale integrated (VLSI) systems, such as system-on-chips (SoCs). More work needs to be done in terms of looking at hardware refactoring, and hardware design patterns.

6 Adoption of Best Practices

Where possible, we shall try to adopt best/good practices from leading product teams (i.e., R&D teams) in the semiconductor and IT industries, and also good researchers spanning electrical engineering and computer science. These practices include: research reproducibility and reproducible research [29, 33, 44, 77, 89, 93, 136, 137, 139, 204, 212, 237, 244, 343, 359, 382], build automation [78, 90, 111, 125, 169, 182, 197, 292, 325, 336, 356–358, 385], distributed version/revision control (or software configuration management) [13, 23, 61, 62, 68, 69, 78, 90, 112, 130, 146–149, 151–153, 171, 182, 250, 252, 293, 298, 299, 317, 325, 355, 357, 366, 387, 390, 415], design by contract (DbC; or, contract programming, programming by contract, or design-by-contract programming) [55, 170, 186, 266, 348, 380, 389, 437] when using the procedural/imperative programming paradigm [26, 37, 233, 240, 245, 271, 306, 332, 364, 396, 409, 424] and Hoare logic [28, 173, 186, 209, 217, 223, 270, 277, 307, 348, 361, 431, 437, 440], regression testing [10, 320, 356, 358, 406], automated software testing [41, 61, 88, 92, 122, 182, 189, 201, 219, 267, 278, 286, 300, 320, 322, 338, 346, 356, 358, 370, 380, 384, 400, 412, 437], and automated regression testing [320, 335]. VLSI verification engineers can use regression verification [7, 8, 10, 26, 48, 58, 155, 236, 309–313, 368, 393] for VLSI system design to check if incremental additions/modifications to the existing IC design components and system would introduce IC design bugs (e.g., functional or security bugs); this is an analogy of regression testing for software. Similarly, endeavor to use the concepts of abstraction [72, 79, 133, 144, 174, 177, 185, 214, 214, 232, 255, 376, 407, 408] and encapsulation [28, 54, 81, 120, 177, 213, 233, 245, 256, 314, 325, 327, 363, 371] with hierarchical design methodologies [9, 60, 125, 168, 210, 211, 283, 345, 401], hierarchical design space exploration [386], top-down hierarchical approach for design steps [73, 107, 132, 135, 180, 194, 241, 351, 413] and bottom-up hierarchical approach for verification steps [72, 132, 255, 323, 408], or rather top-down approach of incremental verification (also known as the modified V approach) [26], and platform-based design [26, 27, 59, 71, 103, 135, 203, 243, 246, 347, 351–354] in our projects involving VLSI design. In addition, use design patterns (Add Citations!!!) to improve VLSI system architecture designs. In general, adopt good practices from software development and data science to improve IC design flows (Add Citations!!!, from my USC report comparing software test automation with VLSI

test automation to a Quora post to job descriptions from that firm paying FPGA design interns at least US\$125 per hour).

In addition, try to use agile (software development and VLSI design) methodologies [1, 15, 17, 18, 22, 25, 34–36, 47, 61, 76, 78, 82, 86, 90, 94, 96, 105, 113, 130, 143, 158, 184, 190, 191, 198, 199, 223, 226–231, 234, 247–249, 284, 308, 318, 320, 324, 339–342, 357, 366, 377, 381, 384, 402, 436] to develop software as well as design electronic circuits and systems [43, 157, 178, 192, 257, 430] [142, Chapter 6, §6.2.2.3, pp. 243], and cyber-physical systems (or embedded systems) [275, 276]. A strong motivation for using these methodologies and their associated practices is to reduce technical debt [1, 50, 78, 167, 215, 280, 326, 339, 379, 395]. In addition, iterative and incremental development (I.I.D.) can help software developers, integrated circuit/system designers, and cyber-physical system designers validate earlier versions (from earlier iterations) of their software builds, and VLSI system and cyber-physical system designs, with (project) stakeholders, repeat this (much) more frequently, and make necessary changes before incurring huge costs with inflexible software development process models or engineering workflows (such as VLSI design and verification flows) for VLSI systems and cyber-physical systems [1, 25, 32, 46, 47, 54, 56, 121, 189, 216, 225–228, 337, 357, 366, 367, 370, 384, 434].

Also, carry out refactoring [123, 129] on an ad-hoc basis to improve the software [1, 25, 38, 57, 90, 106, 109, 116, 119, 130, 190, 200, 207, 253, 265, 366, 372, 380], hardware [117, 118, 175–177, 195, 196, 425, 438, 439], and/or system [78, 222, 339] architecture as well as databases [19]. In terms of personal and professional development, collaborators are strongly encouraged to refactor their wetware [183] and reduce their personal technical debt [2, 95, 134, 235, 315], too.

We shall also use project portfolio management [340, 341] to help us manage projects that we are involved in.

Moreover, for projects involving integrated circuit design (and embedded hardware or cyber-physical system design), morph your process for VLSI CAD engineering into hardware DevOps (hardware/IC development and information-technology operations) [281].

6.1 Practice of Automated Regression Testing

Regarding the practice of automated regression testing, Mr. Heiko Maurer (then a lecturer at the University of Adelaide) and Dr./Mr. Tishampati Dhar (a former classmate at the University of Adelaide) suggests printing information regarding passed test cases to a file (or to standard output) and printing information regarding failed test cases to another file (or to standard error). During build automation of software, such as `gem5` [42, 140], carry out automated (regression) testing during the last stage of the build/installation process to ensure that the build/installation was done correctly. When performing automated software testing (or software test automation), list the the test cases and their test results (i.e., “OK”/“Fail”), just like `gem5` during the testing phase of build automation. At the end of each automated software testing run (or round/run of automated software testing), indicate the total number of test cases used, the total number of test cases passed, and the percentage of test cases passed (with respect to the total number of test cases used).

[391, §Testing Guidelines] provides a set of testing guidelines for *Python* libraries and packages, or software in general.

6.2 Cloud-based Data Science and Machine Learning

Regarding data science projects, we shall use a lot of the aforementioned software development practices and methodologies [61]. Also, we can use cloud-based machine learning (and deep learning) platforms, such as Google’s *Colaboratory* [159], Anaconda’s *Anaconda Cloud* [20], and Amazon’s *Amazon Web Services* (AWS) [16]. These cloud-based software services, also known as software as a service (SaaS), helps to bring data science, machine learning, and deep learning capabilities to more people, since they do not need expensive, modern hardware to run computationally intensive tasks for data analytics and machine learning.

7 Intellectual Property Protection, via Licenses

To be continued...

Add references for:

1. source-available software (get information from Wikipedia)
2. MIT license
3. GNU General Public License, GNU GPL, GPL:
 - (a) GPL version 3, GPLv3
 - (b) GPL version 2, GPLv2
4. Creative Commons (CC) license

8 Additional Guidelines

Please kindly use the *Markdown* language for writing text documents. This is because *Bitbucket* will treat my text file as a file written in the *Markdown* syntax. That said, the raw file looks a lot better than the represented *Markdown* files. Their (*Bitbucket*) formatting for *Markdown* is messed up. *GitHub*’s formatting for *Markdown* works as expected. To insert mathematical expressions into *Markdown* documents, use *TexPaste* [282] to typeset the mathematical expressions via $\text{\LaTeX}$ and insert snapshots of these mathematical expressions as pictures in the *Markdown* documents. A recommended style guide for *Markdown* is from *Google* [399].

In addition, tools for working with source code and $\text{\LaTeX}$ source files include:

1. **git** (or **Git**) [112, 171]:
 - (a) **Mercurial SCM** [24, 101, 260–264, 298] can be used as a substitute.
2. **latexdiff**: “determine and markup differences between two latex files”
 - (a) Evan Driscoll, “Latexdiff notes,” from *Evan Driscoll’s Web page: Writings on Software: $\text{\LaTeX}$* , the Department of Computer Sciences, University of Wisconsin-Madison College of Engineering, University of Wisconsin-Madison, Madison, WI. Available online at: <http://pages.cs.wisc.edu/~driscoll/software/latex/latexdiff.html>; last accessed on February 15, 2016 [110].
3. documentation generators:
 - (a) **Doxygen** [404]
 - (b) **Texinfo**-based generators [373–375, 422]
4. Build automation:
 - (a) **SCons** [111]

Data sets and sets of benchmarks for experiments shall be publicly published using an online repository, via *figshare LLP* [124] and/or *DataHub* [93]. For each data set, or each set of benchmarks, create a unique Digital Object Identifier (DOI) [188] to identify it.

Repositories for software as well as designs of integrated circuits and cyber-physical systems shall be stored online, using online repositories such as *GitHub* [150]. Each repository shall have a unique DOI to identify it, and include all source code, documentation, and design files. There also exists cloud-based repositories for the source code of software/hardware projects that allow me to execute my software (or simulate my hardware). E.g., see [77, 343] as examples to facilitate research reproducibility, replicability, and repeatability. This supports research reproducibility and reproducible research [29, 33, 44, 77, 89, 93, 136, 137, 139, 204, 212, 237, 244, 343, 359, 382].

Please kindly note that *GitHub* [150]:

1. Does not allow a *GitHub*-based page to be refreshed/reloaded many times in a few seconds. Else, it would report the following:
 - (a) “Whoa there!”
 - (b) “You have triggered an abuse detection mechanism.”
 - (c) “Please wait a few minutes before you try again.”

If possible, develop *Python* software in a *Pythonic* style [14, Chapter 1, pp. 1–12, 12–17] [131, 193, 316, 330, 331, 405].

As aforementioned in §2.4, when copying (and pasting) text from a document or a Web page, non-ASCII hidden characters may be accidentally copied from the source (document or Web page) to the source code for a computer language. Similarly, corruption in memory subsystems or storage devices (such as a solid-state drives) and data transfer between computers can result in corrupting text files to produce non-ASCII hidden characters in these files. Therefore, if these text files use a particular computer language, use parsers (for software such as compilers, interpreters, electronic design automation software, or other software applications) for these languages to detect/find these non-ASCII hidden characters.

If text is required to be in the uppercase or capitals, it can still be written as normal, but use the $\text{\LaTeX}$ command `\uppercase` to turn the text within the braces or curly brackets into uppercase. An example is provided as follows: “THIS IS AN EXAMPLE OF TEXT TURNED INTO UPPERCASE” [388, §8.2.4, pp. 239]. Another method is to use the $\text{\LaTeX}$ command `\MakeUppercase` [208, §Appendix G.1, pp. 512], and an example is: “ANOTHER EXAMPLE OF TEXT TURNED INTO UPPERCASE” [166, §6.8, pp. 47; §23.2, pp. 212–213] [272, §2.2.2, pp. 31; §3.1.5, pp. 85–87; §3.1.7, pp. 91; §4.4.2, pp. 229; §9.4.1, pp. 571] [388, §3.5, pp. 60] [435, §5, Changing Letter Case].

Their dual $\text{\LaTeX}$ commands are: `\lowercase` and `\MakeLowercase` [208, §Appendix G.1, pp. 512]. Examples for these are: “this is an example of text turned into lowercase” [388, §8.2.4, pp. 239] and “another example of text turned into lowercase” [166, §23.2, pp. 212–213] [272, §2.2.6, pp. 37; §3.1.5, pp. 85–87; §7.3.1, pp. 341; §9.4.1, pp. 571] [388, §3.5, pp. 60] [435, §5, Changing Letter Case].

8.1 Suggested Guidelines for Benchmarking

Computational notebooks ((Cite this!!!)) based on interactive computing ((Cite this!!!)), such as Jupyter notebooks, are strongly recommended for reproducing statistical analyses of experi-

mental/simulation results that enable research reproducibility [29, 33, 44, 77, 89, 93, 136, 137, 139, 204, 212, 237, 244, 343, 359, 382]. In lieu of

8.2 Suggested Prefixes for Labels of Parts of a L^AT_EX Document

Add one of the following prefixes to labels in my L^AT_EX documents for parts, such as chapters, sections, subsections, subsubsections, figures, tables, equations, code listings, definitions, theorems, lemmas, corollaries, propositions, proofs, examples, and remarks:

1. “chp:” for chapter
2. “sec:” for section
3. “ssec:” for subsection
4. “sssec:” for subsubsection
5. “fig:” for figure
6. “tab:” for table
7. “eqn:” for equation
8. “lst:” for code listing
9. “defn:” for definition
10. “thrm:” for theorem
11. “lem:” for lemma
12. “crly:” for corollary
13. “prop:” for proposition
14. “prf:” for proof
15. “eg:” for example
16. “rem:” for remark

This practice makes it easier for collaborators and paper/reviewers to determine if the label for a part of the L^AT_EX document, belongs to a label or refers to something else. This should be done with cross-referencing different parts of my L^AT_EX documents to make it easier for readers to follow the storyline, and look up information.

Acknowledgments

Mr. David Knight (then a lecturer at the University of Adelaide) and Dr. Charles Lakos (then a senior lecturer at the University of Adelaide) introduced me to regression testing and automated software testing during their introductory course on software engineering. During programming assignments and projects for this course, Dr. Nikolay Stoimenov helped me honed my skills in regression testing and automated software testing, via the practice of pair programming [104, 130, 189, 294, 366, 414]. Subsequently, Mr. Heiko Maurer (then a lecturer at the University of Adelaide) planted the seeds of automated regression testing with his suggestion of separating the results of test cases that passed from the results of test cases that failed. Shortly after, Dr./Mr. Tishampati Dhar (a former classmate at the University of Adelaide) suggests printing information regarding passed test cases to a file (or to standard output) and printing information regarding failed test cases to another file (or to standard error). Months later, Dr. Francis Vaughan (then a senior lecturer at the University of Adelaide), Mr. Kevin J. Maciunas (then a lecturer at the University of Adelaide), and Dr. Robert Esser (then a senior lecturer at the University of Adelaide) helped me develop a sound methodology towards automated regression testing. In addition, Dr. Lakos and Dr. Esser introduced me to using formal methods and software formal verification in the software development process.

One of my favorite directors at a Latinx/Latine/Hispanic organization that I have been part of talked to us about ethical and healthy conflict resolution, during a training session that she ran for our organization. I am keeping her name and organization anonymous to protect our privacy.

To-Do List

To-do list:

1. Update guidelines with references (such as books) on:
 - (a) DevOps:
 - i. Agile DevOps
 - (b) DevSecOps:
 - i. Agile DevSecOps
 - (c) MLOps, ML Ops:
 - i. MLSecOps
 - (d) DataOps:
 - i. Including tools to manage data pipelines:
 - A. Apache Beam:
 - “open-source unified programming model, and set of software development kits (SDKs), to define and execute data processing pipelines, including ETL, batch and stream (continuous) processing. The data pipelines (Beam Pipelines) are executed in one of the supported distributed processing back-ends (Beam supported runners).”
 - ii. DataSecOps
 - (e) ModelOps
 - (f) AIOps
 - (g) workflow management tools:
 - i. Apache Airflow
 - (h) Ignore the following, since it is beyond the scope of what I can do as a researcher or research group leader:
 - i. ITOps
 - ii. IT operations analytics, ITOA
2. incremental computing:
 - (a) https://en.wikipedia.org/wiki/Incremental_computing.
 - (b) Include references for checkpointing, to save intermediate set of simulation results.

References

- [1] 37signals staff. Getting Real: The smarter, faster, easier way to build a successful web application. 37signals, Chicago, IL, 2006.
- [2] Sara Aboulafia. 5 ways to kill your credibility as a product manager. Available online from *UserVoice: UserVoice Blog, The Product Management Blog* at: <https://community.uservoice.com/blog/5-ways-to-kill-your-credibility-as-a-product-manager/>; September 26, 2018 was the last accessed date, July 14 2016.

- [3] David Abrahams. Exception-safety in generic components: Lessons learned from specifying exception-safety for the C++ standard library. In Proceedings of the International Seminar on Generic Programming, volume 1766 of Lecture Notes in Computer Science, pages 69–79, Wadern, Merzig-Wadern, Saarland, Germany, April 27 – May 1 1998. Springer-Verlag Berlin Heidelberg.
- [4] David Abrahams. Exception-safety in generic components: Lessons learned from specifying exception-safety for the C++ standard library. Available online from *Boost C++ Libraries: Community* at: http://www.boost.org/community/exception_safety.html; self-published; October 27, 2016 was the last accessed date, 2001.
- [5] ACM Council. ACM code of ethics and professional conduct. Available online from *Association for Computing Machinery: What is ACM?: Policies and Procedures* at: <http://www.acm.org/about/code-of-ethics>; September 27, 2014 was the last accessed date, October 16 1992.
- [6] ACM Public Policy Office staff. Statement on algorithmic transparency and accountability. Available online from *Association for Computing Machinery: ACM Public Policy Office* at: https://www.acm.org/binaries/content/assets/public-policy/2017_usacm_statement_algorithms.pdf; February 24, 2017 was the last accessed date, January 12 2017.
- [7] John Adler. Novel approaches to automated digital design debugging in a modern development cycle. Master’s thesis, University of Toronto, Toronto, Ontario, Canada, November 2017.
- [8] John Adler and Andreas Veneris. Leveraging software configuration management in automated RTL design debug. IEEE Design & Test, 34(5):47–53, September/October 2017.
- [9] Saurabh N. Adya. Unification of VLSI placement and floorplanning. PhD thesis, University of Michigan, Ann Arbor, MI, 2004.
- [10] David J. Agans. Debugging: The 9 Indispensable Rules for Finding Even the Most Elusive Software and Hardware Problems. AMACOM, New York, NY, 2006.
- [11] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. Compilers: Principles, Techniques, & Tools. Pearson Education, Boston, MA, second edition, 2007.
- [12] Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. Compilers: Principles, Techniques, & Tools. Pearson Education, Reading, MA, 1986.
- [13] Carl Albing, J. P. Vossen, and Cameron Newham. bash Cookbook: Solutions and Examples for bash Users. O’Reilly Media, Sebastopol, CA, 2007.
- [14] Marty Alchin. Pro Python: Advanced Coding Techniques and Tools. The Expert’s Voice® in Open Source. Apress, Berkeley, CA, 2010.
- [15] Rachel Alt-Simmons. Agile by Design: An Implementation Guide to Analytic Lifecycle Management. Wiley & SAS Business. John Wiley & Sons, Hoboken, NJ, 2016.
- [16] Amazon Web Services staff. Amazon Web Services (AWS): Cloud computing services. Available online from *Amazon* at: <https://aws.amazon.com/>; February 11, 2019 was the last accessed date, 2019.
- [17] Scott W. Ambler. Agile Database Techniques: Effective Strategies for the Agile Software Developer. Wiley Publishing, Indianapolis, IN, 2003.

- [18] Scott W. Ambler and Mark Lines. Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise. IBM Press and Pearson PLC, Armonk, NY and Boston, MA, 2012.
- [19] Scott W. Ambler and Pramod J. Sadalage. Refactoring Databases: Evolutionary Database Design. Addison-Wesley Signature Series. Addison-Wesley, Boston, MA, 2006.
- [20] Anaconda staff. Anaconda cloud. Available online at: <https://anaconda.org/>; February 11, 2019 was the last accessed date, 2019.
- [21] Ronald E. Anderson, Deborah G. Johnson, Donald Gotterbarn, and Judith Perrolle. Using the new ACM code of ethics in decision making. Communications of the ACM, 36(2):98–107, February 1993.
- [22] Jurgen Appelo. Management 3.0: Leading Agile Developers, Developing Agile Leaders. Addison-Wesley Signature Series. Addison-Wesley, Boston, MA, 2011.
- [23] Atlassian staff. Tutorial: Learn Bitbucket with Git. Available online from *Atlassian: Atlassian Documentation: Bitbucket Cloud documentation: Tutorials* at: <https://confluence.atlassian.com/bitbucket/tutorial-learn-bitbucket-with-git-759857287.html>; February 15, 2019 was the last accessed date, October 24 2018.
- [24] Atlassian staff. Tutorial: Learn Bitbucket with Mercurial. Available online from *Atlassian: Atlassian Documentation: Bitbucket Cloud documentation: Tutorials* at: <https://confluence.atlassian.com/bitbucket/tutorial-learn-bitbucket-with-mercurial-759857394.html>; February 15, 2019 was the last accessed date, October 24 2018.
- [25] Muhammad Ali Babar, Alan W. Brown, and Ivan Mistrik. Agile Software Architecture: Aligning Agile Processes and Software Architectures. Morgan Kaufmann, Waltham, MA, 2014.
- [26] Brian Bailey and Grant Martin. ESL Models and their Application: Electronic System Level Design and Verification in Practice. Embedded Systems. Springer Science+Business Media, LLC, New York, NY, 2010.
- [27] Brian Bailey, Grant Martin, and Thomas Anderson. Taxonomies for the Development and Verification of Digital Systems. Springer Science+Business Media, Inc., New York, NY, 2005.
- [28] Douglas Baldwin and Greg W. Scragg. Algorithms and Data Structures: The Science of Computing. Computer Engineering Series. Charles River Media, Hingham, MA, 2004.
- [29] Lorena A. Barba. The hard road to reproducibility. Science, 354(6308):142, October 7 2016.
- [30] Michael Barr. Embedded C coding standard. Self-published, Gaithersburg, MD, 2013.
- [31] Wojciech Basalaj and Richard Corden. High integrity C++ coding standard V4.0: An overview. Technical report, Programming Research Ltd., Hersham, Surrey, South East England, England, U.K., November 2013.
- [32] Basecamp staff. Getting real: The smarter, faster, easier way to build a successful web application. Available online from *Basecamp: Books by Basecamp* at: <https://basecamp.com/gettingreal>, <https://basecamp.com/books/getting-real>, and <https://basecamp.com/gettingreal/getting-real.pdf>; February 29, 2020 was the last accessed date, Chicago, IL, 2020.

- [33] Benjamin S. Baumer, Daniel T. Kaplan, and Nicholas J. Horton. Modern Data Science with R. Texts in Statistical Science. CRC Press, Boca Raton, FL, 2017.
- [34] Kent Beck and Cynthia Andres. Extreme Programming Explained: Embrace Change. The XP Series. Pearson Education, Upper Saddle River, NJ, second edition, 2005.
- [35] Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for Agile software development. Available online at: <http://www.agilemanifesto.org/>; February 7, 2016 was the last accessed date, 2001.
- [36] Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Principles behind the agile manifesto. Available online from *Manifesto for Agile Software Development* at: <http://agilemanifesto.org/principles.html>; June 9, 2018 was the last accessed date, 2001.
- [37] Mordechai Ben-Ari. Understanding Programming Languages. John Wiley & Sons, Chichester, West Sussex, South East England, England, U.K., 1996.
- [38] James Bender and Jeff McWherter. Professional Test Driven Development with C#: Developing Real-World Applications with TDD. Wrox Programmer to Programmer™. Wiley Publishing, Indianapolis, IN, 2011.
- [39] Lionel Bening and Harry Foster. Principles of Verifiable RTL Design: A Functional Coding Style Supporting Verification Processes in Verilog. Kluwer Academic Publishers, New York, NY, 2000.
- [40] Lionel Bening and Harry Foster. Principles of Verifiable RTL Design: A functional coding style supporting verification processes in Verilog. Kluwer Academic Publishers, New York, NY, second edition, 2001.
- [41] Jon Bentley. Programming Pearls. Addison-Wesley, Reading, MA, second edition, 2000.
- [42] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *ACM SIGARCH Computer Architecture News*, 39(2):1–7, May 2011.
- [43] David Black, Hemendra Talesara, and Neil Johnson. Using agile techniques with ESL: For better results. Available online from *North American SystemC User's Group (NASCUG): NASCUG 14* at: http://www.nascug.org/events/14th/Agile_ESL.pdf; self-published; September 25, 2018 was the last accessed date, 2010.
- [44] Steve Blackburn and Matthias Hauswirth. Artifact evaluation. Available online from *University of Lugano: Faculty of Informatics: Evaluate Collaboratory, Experimental Evaluation of Software and Systems in Computer Science* at: <http://evaluate.inf.usi.ch/artifacts>; June 25, 2017 was the last accessed date.
- [45] Peter A. Blume. The LabVIEW Style Book. Prentice Hall, Upper Saddle River, NJ, 2007.
- [46] Barry Boehm, Jo Ann Lane, Supannika Koolmanojwong, and Richard Turner. The Incremental Commitment Spiral Model: Principles and Practices for Successful Systems and Software. Addison-Wesley, Upper Saddle River, NJ, 2014.

- [47] Barry Boehm and Richard Turner. Balancing Agility and Discipline: A Guide for the Perplexed. Pearson Education, Boston, MA, 2004.
- [48] Nicola Bombieri, Franco Fummi, and Valerio Guarnieri. FAST: An RTL fault simulation framework based on RTL-to-TLM abstraction. Journal of Electronic Testing, 28(4):495–510, August 2012.
- [49] Tara Brach. Radical Acceptance: Embracing Your Life with the Heart of a Buddha. Bantam Books Trade Paperbacks, New York, NY, 2004.
- [50] Seshu Brahma. The benefits of personal projects. Available online from *The University of Texas at Austin: College of Natural Sciences: Department of Computer Science: ReadMe – UTCS Student Blogs: Seshu Brahma’s blog* at: <http://www.cs.utexas.edu/blog/benefits-personal-projects>; April 17, 2017 was the last accessed date, April 14 2017.
- [51] Bo Brinkman, Catherine Flick, Don Gotterbarn, Keith Miller, Kate Vazansky, and Marty J. Wolf. Listening to professional voices: Draft 2 of the ACM code of ethics and professional conduct. Communications of the ACM, 60(5):105–111, May 2017.
- [52] Bo Brinkman, Don Gotterbarn, Keith Miller, and Marty J. Wolf. Making a positive impact: Updating the ACM code of ethics. Communications of the ACM, 59(12):7–13, December 2016.
- [53] Bo Brinkman, Don Gotterbarn, Keith W. Miller, and Marty J. Wolf. All hands on deck for ACM ethics: Updating the Code, revising enforcement, promoting integrity. ACM Computers and Society: Newsletter of the ACM Special Interest Group on Computers and Society, 46(3):5–8, November 2016.
- [54] Frederick P. Brooks, Jr. The Mythical Man-Month: Essays on Software Engineering. Addison-Wesley, Reading, MA, 20th anniversary edition, 1995.
- [55] Frederick P. Brooks, Jr. The Design of Design: Essays from a Computer Scientist. Pearson Education, Boston, MA, 2010.
- [56] Gregory T. Brown. Programming Beyond Practices: Be More Than Just a Code Monkey. O’Reilly Media, Sebastopol, CA, 2017.
- [57] William J. Brown, Raphael C. Malveau, Hays W. “Skip” McCormick III, and Thomas J. Mowbray. AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis. John Wiley & Sons, Hoboken, NJ, 1998.
- [58] Ian Burgess. Regression Testing: Gate-Level Functional Verification Is Imperative and Equivalence Checking Provides the Solution. Mentor Graphics Corporation, Wilsonville, OR, 2004.
- [59] Mark Burton and Adam Morawiec. Platform Based Design at the Electronic System Level: Industry Perspectives and Experiences. Springer, Dordrecht, The Netherlands, 2006.
- [60] Giorgio C. Buttazzo. Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, volume 24 of Real-Time Systems Series. Springer Science+Business Media, LLC, New York, NY, third edition, 2011.
- [61] Field Cady. The Data Science Handbook. John Wiley & Sons, Hoboken, NJ, 2017.

- [62] Luiz Fernando Capretz and Miriam A. M. Capretz. Object-Oriented Software Design and Maintenance, volume 6 of Series on Software Engineering and Knowledge Engineering. World Scientific Publishing, Singapore, 1996.
- [63] Frank Carbajal and Humberto Medina. Building the Latino Future: Success Stories for the Next Generation. John Wiley & Sons, Hoboken, NJ, 2008.
- [64] Tom Cargill. C++ Programming Style. Addison-Wesley, Reading, MA, 1992.
- [65] Dale Carnegie. How To Win Friends and Influence People. Simon & Schuster, New York, NY, 1936.
- [66] Frank M. Carrano. Data Structures and Abstractions with Java. Prentice Hall, Upper Saddle River, NJ, third edition, 2012.
- [67] Eric Carter and Matthew Hurst. Agile Machine Learning: Effective Machine Learning Inspired by the Agile Manifesto. Apress Media, LLC, Berkeley, CA, 2019.
- [68] Scott Chacon. Pro Git. Apress, New York, NY, 2009.
- [69] Scott Chacon and Ben Straub. Pro Git: Everything You Need to Know About Git. The Expert's Voice®. Apress Media, LLC, Berkeley, CA, second edition, 2014.
- [70] Candy Chang. Before I Die. St. Martin's Press, New York, NY, 2013.
- [71] Henry Chang, Larry Cooke, Merrill Hunt, Grant Martin, Andrew McNelly, and Lee Todd. Surviving the SoC Revolution: A Guide to Platform-Based Design. Kluwer Academic Publishers, New York, NY, 2002.
- [72] Pankaj P. Chauhan. Verification of Large Industrial Circuits Using SAT Based Reparameterization and Automated Abstraction-Refinement. PhD thesis, Carnegie Mellon University, Pittsburgh, PA, May 2007.
- [73] Jess Chen, Michael Henrie, Monte F. Mar, and Mladen Nizic. Mixed-Signal Methodology Guide: Advanced Methodology for AMS IP and SoC Design, Verification, and Implementation. Lulu Enterprises, Raleigh, NC, August 2012.
- [74] Brian Christian. The Alignment Problem: Machine Learning and Human Values. W. W. Norton & Company, New York, NY, 2020.
- [75] Henry Cloud. Boundaries for Leaders: Results, Relationships, and Being Ridiculously in Charge – Why Some People Get Results and Others Don't. HarperBusiness, New York, NY, 2013.
- [76] Alistair Cockburn. Agile Software Development: The Cooperative Game. Agile Software Development Series. Pearson Education, Boston, MA, second edition, 2007.
- [77] Code Ocean staff. Code ocean: Discover & run scientific code. Available online at: <https://codeocean.com>; February 2, 2018 was the last accessed date, 2017.
- [78] Ken Collier. Agile Analytics: A Value-Driven Approach to Business Intelligence and Data Warehousing. Agile Software Development Series. Addison-Wesley, Boston, MA, 2012.
- [79] Jason Cong and Joseph R. Shinnerl. Multilevel Optimization in VLSICAD, volume 14 of Combinatorial Optimization. Kluwer Academic Publishers, New York, NY, 2003.

- [80] Roger Connors and Tom Smith. Change the Culture, Change the Game: The Breakthrough Strategy for Energizing Your Organization and Creating Accountability for Results. Penguin Group, New York, NY, 2011.
- [81] Jon Conway and Steve Watts. A Software Engineering Approach to LabVIEW™. National Instruments Virtual Instrumentation Series. Prentice Hall, Upper Saddle River, NJ, 2003.
- [82] James O. Coplien and Neil B. Harrison. Organizational Patterns of Agile Software Development 2004. Prentice-Hall, Upper Saddle River, NJ, 2005.
- [83] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms. The MIT Press, Cambridge, MA, second edition, 2001.
- [84] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms. The MIT Press, Cambridge, MA, third edition, 2009.
- [85] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms. The MIT Press, Cambridge, MA, fourth edition, 2022.
- [86] Lawrence Corr and Jim Stagnitto. Agile Data Warehouse Design: Collaborative Dimensional Modeling, from Whiteboard to Star Schema 2011. DecisionOne Press, Leeds, West Yorkshire, Yorkshire and the Humber, England, U.K., 2011.
- [87] Victor Costan, Titus Winters, Ted Osborne, Wei Zhang, Tom Manshreck, Stephen Hicks, Mark Mentovai, Geoff Romer, frp, Alan Yee, and Ackermann Yuriy. Google C++ style guide. Available online from *GitHub: Google: styleguide* at: <https://google.github.io/styleguide/cppguide.html> and <https://github.com/google/styleguide/blob/gh-pages/cppguide.html>; September 17, 2019 was the last accessed date, September 5 2018.
- [88] Coverity staff. Coverity scan: 2012 open source report. Technical report, Coverity, Inc., San Francisco, CA, 2012.
- [89] John W. Creswell. Research Design: Qualitative, Quantitative, and Mixed Methods Approaches. SAGE Publications, Thousand Oaks, CA, fourth edition, 2014.
- [90] Edward Crookshanks. Practical Software Development Techniques: Tools and Techniques for Building Enterprise Software. Nokol Services, third edition, 2012.
- [91] Bonnie Berman Cushing, Lila Cabbil, Margery Freeman, Jeff Hitchcock, and Kimberley Richards. Accountability and White Anti-racist Organizing: Stories from Our Work. Crandall, Dostie & Douglass Books, Inc., Roselle, NJ, 2010.
- [92] Aristides Dasso and Ana Funes. Verification, Validation and Testing in Software Engineering. Idea Group Publishing, Hershey, PA, 2007.
- [93] Datopian (Atomatic Ltd), Inc. staff. DataHub: Frictionless data, data online made simple. Available online at: <http://datahub.io>; February 2, 2018 was the last accessed date, 2017.
- [94] Rachel Davies and Liz Sedley. Agile Coaching. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, 2009.

- [95] Chad Davis. Mental hygiene for software engineers: Avoiding micro level technical debt. Available online from *DZone* at: <https://dzone.com/articles/mental-hygiene-software>; September 27, 2018 was the last accessed date, January 23 2013.
- [96] Andrea De Lucia, Filomena Ferrucci, Genny Tortora, and Maurizio Tucci. Emerging Methods, Technologies, and Process Management in Software Engineering. John Wiley & Sons, Hoboken, NJ, 2008.
- [97] J. Roberto de Marca, E. James Prendergast, Howard E. Michel, Jacek M. Zurada, Gianluca Setti, Ralph M. Ford, Saurabh Sinha, Karen Bartleson, Gary L. Blank, Marko Delimar, John T. Barr, Peter W. Staecker, Ellen J. Yoffa, Jerry L. Hudgins, Harvey A. Freeman, Josef W. Modelski, Susan K. (Kathy) Land, Bogdan M. Wilamowski, Wanda K. Reder, Roger U. Fujii, Marina Ruggieri, Stephen Yurkovich, Vincent P. Socci, Parviz Famouri, Mary Ellen Randall, Karen S. Pedersen, J. Derald Morgan, Michael R. Andrews, Amir G. Aghdam, Martin J. Bastiaans, Norberto Leren-degui, Toshio Fukuda, Eric Herz, and Theodore W. Hissey. IEEE code of conduct. Available online from *IEEE: About IEEE: IEEE Board of Directors Resources: IEEE Governing Documents: IEEE Code of Ethics* at: http://www.ieee.org/about/ieee_code_of_conduct.pdf; October 2, 2014 was the last accessed date, June 2014.
- [98] J. Roberto de Marca, E. James Prendergast, Howard E. Michel, Jacek M. Zurada, Gianluca Setti, Ralph M. Ford, Saurabh Sinha, Karen Bartleson, Gary L. Blank, Marko Delimar, John T. Barr, Peter W. Staecker, Ellen J. Yoffa, Jerry L. Hudgins, Harvey A. Freeman, Josef W. Modelski, Susan K. (Kathy) Land, Bogdan M. Wilamowski, Wanda K. Reder, Roger U. Fujii, Marina Ruggieri, Stephen Yurkovich, Vincent P. Socci, Parviz Famouri, Mary Ellen Randall, Karen S. Pedersen, J. Derald Morgan, Michael R. Andrews, Amir G. Aghdam, Martin J. Bastiaans, Norberto Leren-degui, Toshio Fukuda, Eric Herz, and Theodore W. Hissey. IEEE code of ethics. In *IEEE Policies*, page 7.3. Institute of Electrical and Electronics Engineers, New York, NY, 2014.
- [99] J. Roberto de Marca, E. James Prendergast, Howard E. Michel, Jacek M. Zurada, Gianluca Setti, Ralph M. Ford, Saurabh Sinha, Karen Bartleson, Gary L. Blank, Marko Delimar, John T. Barr, Peter W. Staecker, Ellen J. Yoffa, Jerry L. Hudgins, Harvey A. Freeman, Josef W. Modelski, Susan K. (Kathy) Land, Bogdan M. Wilamowski, Wanda K. Reder, Roger U. Fujii, Marina Ruggieri, Stephen Yurkovich, Vincent P. Socci, Parviz Famouri, Mary Ellen Randall, Karen S. Pedersen, J. Derald Morgan, Michael R. Andrews, Amir G. Aghdam, Martin J. Bastiaans, Norberto Leren-degui, Toshio Fukuda, Eric Herz, and Theodore W. Hissey. IEEE policies: 2014. Technical report, Institute of Electrical and Electronics Engineers, New York, NY, August 2014.
- [100] J. Roberto de Marca, E. James Prendergast, Howard E. Michel, Jacek M. Zurada, Gianluca Setti, Ralph M. Ford, Saurabh Sinha, Karen Bartleson, Gary L. Blank, Marko Delimar, John T. Barr, Peter W. Staecker, Ellen J. Yoffa, Jerry L. Hudgins, Harvey A. Freeman, Josef W. Modelski, Susan K. (Kathy) Land, Bogdan M. Wilamowski, Wanda K. Reder, Roger U. Fujii, Marina Ruggieri, Stephen Yurkovich, Vincent P. Socci, Parviz Famouri, Mary Ellen Randall, Karen S. Pedersen, J. Derald Morgan, Michael R. Andrews, Amir G. Aghdam, Martin J. Bastiaans, Norberto Leren-degui, Toshio Fukuda, Eric Herz, and Theodore W. Hissey. Nondiscrimination policy. In *IEEE Policies*, page 9.8. Institute of Electrical and Electronics Engineers, New York, NY, 2014.
- [101] Mathias De Maré and other Mercurial contributors. Mercurial: the definitive guide. Available online from *Mercurial community: Mercurial SCM: Mercurial – the definitive guide* as version/build 1.90-ebb51bf94742 at: <https://book.mercurial-scm.org/read/> and <https://book.mercurial-scm.org/>; self-published; February 15, 2019 was the last accessed date, 2015.

- [102] Richard Delgado. Art Proverbs. Xlibris Corporation, Bloomington, IN, 2017.
- [103] Douglas Densmore and Abhijit Davare. A Platform-Based Design Methodology for the Electronic System Level: Frameworks, Design Flows, and Case Studies. VDM Verlag Dr. Müller, Saarbrücken, Germany, 2008.
- [104] Andrew DeOrio and Andrew Giugliano. Long-term effects of partner programming in an introductory computer science sequence. In Proceedings of the 123rd ASEE Annual Conference and Exposition on Engineering Education, number 15189 in ASEE Conferences and Meetings, New Orleans, LA, June 26–29 2016. American Society for Engineering Education, American Society for Engineering Education.
- [105] Esther Derby and Diana Larsen. Agile Retrospectives: Making Good Teams Great. The Pragmatic Programmers, Raleigh, NC, 2006.
- [106] Davide Di Gennaro. Advanced C++ metaprogramming. Self-published, 2011.
- [107] Alex Doboli and Edward H. Currie. Introduction to Mixed-Signal, Embedded Design. Springer Science+Business Media, LLC, New York, NY, 2011.
- [108] John Doerr. Measure What Matters: How Google, Bono, and the Gates Foundation Rock the World with OKRs. Portfolio/Penguin, New York, NY, 2018.
- [109] John F. Dooley. Software Development, Design and Coding: With Patterns, Debugging, Unit Testing, and Refactoring. Apress Media, LLC, Berkeley, CA, second edition, 2017.
- [110] Evan Driscoll. Latexdiff notes. Available online from *University of Wisconsin-Madison: University of Wisconsin-Madison College of Engineering: Department of Computer Sciences: Evan Driscoll's Web page: Writings on Software: L^AT_EX* at: <http://pages.cs.wisc.edu/~driscoll/software/latex/latexdiff.html>; February 15, 2016 was the last accessed date.
- [111] Evan Driscoll. Scons documentation recommendatitons. Available online from *University of Wisconsin-Madison: University of Wisconsin-Madison College of Engineering: Department of Computer Sciences: Evan Driscoll's Web page: Writings on Software: SCons* at: <http://pages.cs.wisc.edu/~driscoll/software/scons/index.html>; February 15, 2016 was the last accessed date.
- [112] Evan Driscoll. Version control. Available online from *University of Wisconsin-Madison: University of Wisconsin-Madison College of Engineering: Department of Computer Sciences: Evan Driscoll's Web page: Writings on Software* at: <http://pages.cs.wisc.edu/~driscoll/software/vcs/>; February 15, 2016 was the last accessed date, February 22 2012.
- [113] Jutta Eckstein. Agile Software Development with Distributed Teams: Staying Agile in a Global World. Dorset House Publishing, New York, NY, 2010.
- [114] Maria S. Eitel. A letter to the class of covid-19, from the founder of the nike foundation. Available online from *Marie Claire: Money & Career* and *Yahoo! News* at: <http://www.marieclaire.com/career-advice/a32452725/maria-eitel-letter-graduating-class-2020/> and <https://news.yahoo.com/letter-class-covid-19-founder-180000935.html>; November 17, 2020 was the last accessed date, May 13 2020.

- [115] Benny Evangelista. Facebook's hacker way – “move fast and break things”. Available online from *The San Francisco Chronicle: The SF Gate: Blogs at SFGate.com: The Technology Chronicles* at: <http://blog.sfgate.com/techchron/2012/02/01/facebooks-hacker-way-move-fast-and-break-things/>; October 9, 2014 was the last accessed date, February 1 2012.
- [116] Eric Evans. Domain-Driven Design: Tackling Complexity in the Heart of Software. Pearson Education, Boston, MA, 2004.
- [117] Philippe Faes. What is hardware refactoring? Available online from *Sigasi: Sigasi Insights: Tech Articles* at: http://insights.sigasi.com/tech/hardware_refactoring.html; September 26, 2018 was the last accessed date, December 18 2008.
- [118] Philippe Paul Henri Faes and Hendrik Richard Pieter Eeckhaut. Device and method for refactoring hardware code. Available online from *Google Patents* at: <https://patents.google.com/patent/EP2175387A1/en>; September 26, 2018 was the last accessed date, April 14 2010.
- [119] Michael C. Feathers. Working Effectively with Legacy Code. Robert C. Martin. Pearson Education, Upper Saddle River, NJ, 2005.
- [120] Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, and Shriram Krishnamurthi. How to Design Programs: An Introduction to Programming and Computing. The MIT Press, Cambridge, MA, 2001.
- [121] Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, and Shriram Krishnamurthi. How to Design Programs: An Introduction to Programming and Computing. The MIT Press, Cambridge, MA, second edition, 2018.
- [122] Mark Fewster and Dorothy Graham. Software Test Automation: Effective Use of Test Execution Tools. ACM Press and Addison-Wesley, New York, NY and Harlow, Essex, East of England, England, U.K., 1999.
- [123] Jay Fields, Shane Harvie, Martin Fowler, and Kent Beck. Refactoring. Addison-Wesley Signature Series. Addison-Wesley, Boston, MA, ruby edition, 2010.
- [124] figshare LLP staff. figshare: credit for all your research. Available online at: <https://figshare.com/>; November 4, 2016 was the last accessed date, 2016.
- [125] Michael Fingeroff. High-Level Synthesis Blue Book. Xlibris Corporation, Bloomington, IN, 2010.
- [126] Jonathan D. Fitzgerald. With a little help from my friends. Available online from *Inside Higher Ed: Higher Education Blogs: Blog U: GradHacker – A Blog from GradHacker and MATRIX: The Center for Humane Arts, Letters and Social Sciences Online* at: <https://www.insidehighered.com/blogs/gradhacker/little-help-my-friends>; December 2, 2015 was the last accessed date, December 1 2015.
- [127] Rachel Fong. Move fast and break things. Available online from *Massachusetts Institute of Technology: MIT Admissions: Blogs: Academics & Research* at: http://mitadmissions.org/blogs/entry/move_fast_and_break_things; October 9, 2014 was the last accessed date, June 25 2011.
- [128] James J. F. Forest and Philip G. Altbach. International Handbook of Higher Education, volume 18 of Springer International Handbooks of Education. Springer, Dordrecht, The Netherlands, 2007.

- [129] Martin Fowler, Kent Beck, John Brant, William Opdyke, and Don Roberts. Refactoring: Improving the Design of Existing Code. Addison-Wesley Object Technology Series. Addison-Wesley, Reading, MA, 1999.
- [130] Armando Fox and David Patterson. Engineering Long-Lasting Software: An Agile Approach Using SaaS and Cloud Computing. Strawberry Canyon LLC, beta 0.9.1 edition, January 10 2013.
- [131] Daniel França. C++ has become more Pythonic-Jeff Preshing. Available online from *Standard C++: Blog* at: <https://isocpp.org/blog/2014/12/c-has-become-more-pythonic>; self-published; February 1, 2018 was the last accessed date, December 2 2014.
- [132] Ronny Frevert, Joachim Haase, Roland Jancke, Uwe Knöchel, Peter Schwarz, Ralf Kakerow, and Mohsen Darianian. Modeling and Simulation for RF System Design. Springer, Dordrecht, The Netherlands, 2005.
- [133] Steve Furber. ARM System-on-Chip Architecture. Pearson Education, Harlow, Essex, East of England, England, U.K., second edition, 2000.
- [134] Brian Gadoury. Keep your tools sharp to avoid personal technical debt. Available online from *End Point: Our Blog* at: <https://www.endpoint.com/blog/2010/10/12/keep-your-tools-sharp-to-avoid-personal>; September 26, 2018 was the last accessed date, October 12 2010.
- [135] Daniel D. Gajski, Samar Abdi, Andreas Gerstlauer, and Gunar Schirner. Embedded System Design: Modeling, Synthesis and Verification. Springer Science+Business Media, LLC, New York, NY, 2009.
- [136] Christopher Gandrud. Reproducible Research with R and RStudio. Chapman & Hall/CRC The R Series. CRC Press, Boca Raton, FL, 2014.
- [137] Christopher Gandrud. Reproducible Research with R and RStudio. Chapman & Hall/CRC The R Series. CRC Press, Boca Raton, FL, second edition, 2015.
- [138] Loren Gary, Gary Klein, Ron Ashkenas, Melissa Raffoni, Tom Cross, Jon R Katzenbach, Douglas K. Smith, Nadim F. Matta, Ronald N. Ashkenas, Ray Sheen, Clayton M. Christensen, Matt Marx, Howard H. Stevenson, and Jimmy Guterman. HBR Guide to Project Management: Motivate your team. Avoid scope creep. Deliver results. Harvard Business Review Guides. Harvard Business Review Press, Boston, MA, 2012.
- [139] Matthias Geier. Reproducible research. Available online from *Matthias Geier's Web page* at: http://mg.readthedocs.io/reproducible_research.html; self-published; April 24, 2017 was the last accessed date.
- [140] gem5 developers. The gem5 simulator system: A modular platform for computer system architecture research. Available online at: http://www.gem5.org/Main_Page; self-published; October 9, 2014 was the last accessed date, October 27 2014.
- [141] William Gentry. Be the Boss Everyone Wants to Work For: A Guide for New Leaders. Berrett-Koehler Publishers and Center for Creative Leadership, Oakland, CA and Greensboro, NC, 2016.
- [142] Andreas Gerstlauer, Rainer Dömer, Junyu Peng, and Daniel D. Gajski. System Design: A Practical Guide with SpecC. Kluwer Academic Publishers, Norwell, MA, 2001.

- [143] Imran Ghani, Dayang Norhayati Abang Jawawi, Siva Dorairaj, and Ahmed Sidky. Emerging Innovations in Agile Software Development. Advances in Systems Analysis, Software Engineering, and High Performance Computing. Information Science Reference, Hershey, PA, 2016.
- [144] Joseph (Yossi) Gil and David H. Lorenz. Design patterns and language design. IEEE Computer, 31(3):118–120, March 1998.
- [145] Al Gini and Ronald M. Green. 10 Virtues of Outstanding Leaders: Leadership and Character. Foundations of Business Ethics. Wiley-Blackwell, Chichester, West Sussex, South East England, England, U.K., 2013.
- [146] Git contributors. Community. Available online from *Git* at: <http://www.git-scm.com/community>; August 1, 2014 was the last accessed date, 2014.
- [147] Git contributors. Documentation. Available online from *Git: Documentation* at: <http://www.git-scm.com/doc>; August 1, 2014 was the last accessed date, 2014.
- [148] Git contributors. Reference. Available online from *Git: Documentation: Reference: Reference Manual* at: <http://www.git-scm.com/docs>; August 1, 2014 was the last accessed date, 2014.
- [149] GitHub staff. Git reference. Available online at: <http://gitref.org/>; August 1, 2014 was the last accessed date, 2014.
- [150] GitHub staff. GitHub. Available online at: <https://github.com/>; November 4, 2016 was the last accessed date, 2016.
- [151] GitHub Training Team. GitHub Git cheat sheet. Available online from *GitHub: Training: GitHub Training: GitHub Training Kit: Cheat Sheets: English* at: <https://training.github.com/kit/downloads/github-git-cheat-sheet.pdf>; August 1, 2014 was the last accessed date, 2014.
- [152] GitHub Training Team. GitHub training. Available online from *GitHub: Training* at: <https://training.github.com/>; August 14, 2014 was the last accessed date, 2014.
- [153] GitHub Training Team. GitHub training kit: Open source teaching resources – open source training content. present your way to better version control. Available online from *GitHub: Training: GitHub Training* at: <https://training.github.com/kit/>; August 14, 2014 was the last accessed date, 2014.
- [154] Malcolm Gladwell. Talking to Strangers: What We Should Know About the People We Don't Know. Little, Brown & Company, New York, NY, 2019.
- [155] Marcelo Glusman, Gila Kamhi, Sela Mador-Haim, Ranan Fraer, and Moshe Y. Vardi. Multiple-counterexample guided iterative abstraction refinement: An industrial evaluation. In Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2003), volume 2619 of Lecture Notes in Computer Science, pages 176–191, Warsaw, Poland, April 7–11 2003. European Association for Theoretical Computer Science (EATCS), European Association for Programming Languages and Systems (EAPLS), European Association of Software Science and Technology (EASST), ACM Special Interest Group on Software Engineering (SIGSOFT), ACM Special Interest Group on Programming Languages (SIGPLAN), and ACM Special Interest Group on Algorithms and Computation Theory (SIGACT), Springer-Verlag Berlin Heidelberg.

- [156] Seth Godin. *Small is the New Big: and 183 Other Riffs, Rants, and Remarkable Business Ideas*. Portfolio, New York, NY, 2006.
- [157] Richard Goering. Can agile software development methods help SoC design? Available online from *Cadence Design Systems: Cadence Community: Blogs: Industry Insights Blogs* at: https://community.cadence.com/cadence_blogs_8/b/ii/posts/can-agile-software-development-methods-help-soc-design; September 25, 2018 was the last accessed date, August 18 2010.
- [158] Luis Gonçalves and Ben Linders. Getting value out of agile retrospectives: A toolbox of retrospective exercises. Self-published, Raleigh, NC, September 12 2014.
- [159] Google Colab staff. Colaboratory. Available online from *Google: Google AI: Research: Google Colab* at: <https://colab.research.google.com/>; February 13, 2019 was the last accessed date, 2019.
- [160] Google styleguide contributors. styleguide: Google style guides. Available online from *GitHub: Google* at: <http://google.github.io/styleguide/> and <https://github.com/google/styleguide>; September 17, 2019 was the last accessed date, September 10 2019.
- [161] Don Gotterbarn, Bo Brinkman, Catherine Flick, Michael S. Kirkpatrick, Keith Miller, Kate Varansky, Marty J. Wolf, Eve Anderson, Ron Anderson, Amy Bruckman, Karla Carter, Michael Davis, Penny Duquenoy, Jeremy Epstein, Kai Kimppa, Lorraine Kisselburgh, Shrawan Kumar, Andrew McGettrick, Natasa Milic-Frayling, Denise Oram, Simon Rogerson, David Shama, Janice Sipior, Eugene Spafford, and Les Waguespack. ACM code of ethics and professional conduct. Available online from *Association for Computing Machinery: About ACM: Guiding Members with a Framework of Ethical Conduct: ACM Committee on Professional Ethics* at: <https://www.acm.org/code-of-ethics>; September 19, 2018 was the last accessed date, June 22 2018.
- [162] Don Gotterbarn, Bo Brinkman, Catherine Flick, Keith Miller, Marty Wolf, Kate Vazansky, Florence Appel, Karla Carter, Fran Grodzinsky, Michael S. Kirkpatrick, Anthony Lobo, Denise Oram, and Simon Rogerson. ACM code of ethics and professional conduct. Available online from *Association for Computing Machinery: About ACM: Guiding Members with a Framework of Ethical Conduct: ACM Committee on Professional Ethics* at: <https://ethics.acm.org>; September 19, 2018 was the last accessed date, June 22 2018.
- [163] Don Gotterbarn, Amy Bruckman, Catherine Flick, Keith Miller, and Marty J. Wolf. ACM code of ethics: A guide for positive action. *Communications of the ACM*, 61(1):121–128, January 2018.
- [164] Don Gotterbarn, Keith Miller, and Simon Rogerson. Software engineering code of ethics. *Communications of the ACM*, 40(11):110–118, November 1997.
- [165] Donald Gotterbarn, Keith Miller, Simon Rogerson, Steve Barber, Peter Barnes, Ilene Burnstein, Michael Davis, Amr El-Kadi, N. Ben Fairweather, Milton Fulghum, N. Jayaram, Tom Jewett, Mark Kanko, Ernie Kallman, Duncan Langford, Joyce Currie Little, Ed Mechler, Manuel J. Norman, Douglas Phillips, Peter Ron Prinzivalli, Patrick Sullivan, John Weckert, Vivian Weil, S. Weisband, and Laurie Honour Werth. The joint ACM/IEEE-CS software engineering code of ethics and professional practice. Available online from *Association for Computing Machinery: Committee on Professional Ethics (COPE): ACM Code of Ethics and Professional Conduct: ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices (SEPP)* at: <https://>

- [//ethics.acm.org/code-of-ethics/software-engineering-code/](http://ethics.acm.org/code-of-ethics/software-engineering-code/); September 19, 2018 was the last accessed date, 1999.
- [166] George D. Greenwade, Stephen Gilmore, Torsten Martinsen, and Karl Berry. LaTeX2e unofficial reference manual (may 2022). Available online from *GNU operating system: Repository Listing: latexrefman: trunk* at: <https://latexref.xyz/> and <https://latexref.xyz/dev/latex2e.pdf>; self-published; June 4, 2022 was the last accessed date, May 13 2022.
 - [167] Yuepu Guo. Measuring and Monitoring Technical Debt. PhD thesis, University of Maryland, Baltimore County, Baltimore, MD, 2016.
 - [168] Rajesh Gupta, Paul Le Guernic, Sandeep Kumar Shukla, and Jean-Pierre Talpin. Formal Methods and Models for System Design: A System Level Perspective. Springer Science+Business Media Dordrecht, Dordrecht, The Netherlands, 2004.
 - [169] Philip N. Hagelberg. Leiningen. Available online from *GitHub: Phil Hagelberg* at: <https://github.com/technomancy/leiningen>; October 23, 2017 was the last accessed date, October 21 2017.
 - [170] Max Hailperin, Barbara Kaiser, and Karl Knight. Concrete Abstractions: An Introduction to Computer Science Using Scheme. Brooks/Cole, Pacific Grove, CA, 1999.
 - [171] Junio C. Hamano, Linus Torvalds, Shawn Pearce, Johannes Schindelin, Nicolas Pitre, René Scharfe, Jeff King, Jonathan Nieder, Johan Herland, Johannes Sixt, Sverre Rabbelier, Michael J. Gruber, Nguyễn Thái Ngọc Duy, Ævar Arnfjörð Bjarmason, and Thomas Rast. Git. Available online at: <http://www.git-scm.com/> and <http://git-blame.blogspot.com/>; August 1, 2014 was the last accessed date, August 12 2014.
 - [172] Rick Hanson. Just One Thing: Developing a Buddha Brain One Simple Practice at a Time. New Harbinger Publications, Oakland, CA, 2011.
 - [173] David Harel, Dexter Kozen, and Jerz Tiuryn. Dynamic Logic. Foundations of Computing. The MIT Press, Cambridge, MA, 2000.
 - [174] David Money Harris and Sarah L. Harris. Digital Design and Computer Architecture. Morgan Kaufmann, Waltham, MA, 2007.
 - [175] Steve Haynal. Refactoring to prepare RTL for reuse. Available online from *SofterHardware: Publications* at: <http://www.softerhardware.com/pdf/ip08a.pdf>; self-published; September 26, 2018 was the last accessed date, 2008.
 - [176] Steve Haynal. Refactoring to prepare RTL for reuse. Available online from *Design & Reuse: D&R Industry Articles* at: <https://www.design-reuse.com/articles/20119/refactoring-rtl-reuse.html>; linked from <https://www.design-reuse.com/articles/archive200902.html>; September 26, 2018 was the last accessed date, February 16 2009.
 - [177] Steve Haynal, Timothy Kam, Michael Kishinevsky, Emily Shriver, and Xinning Wang. A SystemVerilog rewriting system for RTL abstraction with Pentium case study. In Proceedings of the 6th ACM/IEEE International Conference on Formal Methods and Models for Co-Design (MEMOCODE '08), pages 79–88, Anaheim, CA, June 5–7 2008. ACM Special Interest Group on Design Automation (SIGDA), ACM Special Interest Group on Embedded Systems (SIGBED), IEEE Circuits and Systems Society (CAS), and IEEE Council on EDA (CEDA), IEEE Press.

- [178] John Hennessy and David Patterson. A new golden age for computer architecture: Domain-specific hardware/software co-design, enhanced security, open instruction sets, and agile chip development. Available online from *IEEE Consultants' Network of Silicon Valley (IEEE-CNSV): Events* at: <https://californiaconsultants.org/event/annual-cnsv-dinner-meeting-a-new-golden-age-for-computer-architecture/> and <https://californiaconsultants.org/wp-content/uploads/2018/04/CNSV-1806-Patterson.pdf>; July 20, 2018 was the last accessed date, June 13 2018.
- [179] Hewlett-Packard Company. Dave packard's 11 simple rules. Available online from *Hewlett-Packard Company: Retiree Home: History: Founders & early contributors-Dave Packard* at: <http://www.hp.com/retiree/history/founders/packard/11rules.html>; March 15, 2014 was the last accessed date, 2012.
- [180] Tsung-Yi Ho, Yao-Wen Chang, and Sao-Jie Chen. Full-Chip Nanometer Routing Techniques. Analog Circuits and Signal Processing. Springer, Dordrecht, The Netherlands, 2007.
- [181] Todd Hoff. C++ coding standard. Available online from *Possibility Outpost: Programmer's Corner* at: <http://www.possibility.com/Cpp/CppCodingStandard.html>; February 16, 2016 was the last accessed date, March 1 2008.
- [182] Jez Humble and David Farley. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Signature Series. Pearson Education, Boston, MA, 2011.
- [183] Andy Hunt. Pragmatic Thinking and Learning: Refactor Your Wetware. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, September 15 2008.
- [184] John Hunt. Agile Software Construction. Springer-Verlag London, London, U.K., 2006.
- [185] Mohamed Hamed Zaki Hussein. Techniques for the Formal Verification of Analog and Mixed-Signal Designs. PhD thesis, Concordia University, Montréal, Québec, Canada, 2008.
- [186] Michael Huth and Mark Ryan. Logic in Computer Science: Modelling and Reasoning about Systems. Cambridge University Press, New York, NY, second edition, 2004.
- [187] Intel Corporation staff. Intel® 64 and IA-32 Architectures Optimization Reference Manual. Intel Corporation, Santa Clara, CA, September 2015.
- [188] International DOI Foundation staff. Digital object identifier system. Available online at: <https://www.doi.org/>; January 20, 2017 was the last accessed date, January 10 2017.
- [189] Pankaj Jalote. A Concise Introduction to Software Engineering. Undergraduate Topics in Computer Science. Springer-Verlag London, London, U.K., 2008.
- [190] Ron Jeffries. The Nature of Software Development: Keep It Simple, Make It Valuable, Build It Piece by Piece. The Pragmatic Programmers, Raleigh, NC, 2015.
- [191] Randall W. Jensen. Improving Software Development Productivity: Effective Leadership and Quantitative Methods in Software Management. Prentice Hall, Upper Saddle River, NJ, 2015.
- [192] Neil Johnson and Bryan Morris. AgileSoC: Bring agile to the world of hardware development. Available online at: <http://agilesoc.com>; self-published; September 25, 2018 was the last accessed date, 2018.

- [193] Jon. What does pythonic mean? [closed]. Available online from *Stack Exchange Inc.: Stack Overflow: Questions* at: <https://stackoverflow.com/questions/25011078/what-does-pythonic-mean>; February 1, 2018 was the last accessed date, July 29 2014.
- [194] Hubert Kaeslin. Top-Down Digital VLSI Design: From Architectures to Gate-Level Circuits and FPGAs. Morgan Kaufmann, Waltham, MA, 2015.
- [195] Michael Keating and Pierre Bricaud. Reuse Methodology Manual: For System-on-a-Chip Designs. Kluwer Academic Publishers, Dordrecht, The Netherlands, second edition, 1999.
- [196] Michael Keating and Pierre Bricaud. Reuse Methodology Manual: For System-on-a-Chip Designs. Kluwer Academic Publishers, Dordrecht, The Netherlands, third edition, 2002.
- [197] Eric R. Keiter, Thomas V. Russo, Richard L. Schiek, Peter E. Sholander, Heidi K. Thornquist, Ting Mei, Jason C. Verley, and David G. Baur. Building guide: How to build Xyce™ from source code. Technical Report SAND2013-7294, Sandia National Laboratories, Sandia Corporation, Albuquerque, NM, August 2013.
- [198] Allan Kelly. Changing Software Development: Learning to Become Agile. John Wiley & Sons, Chichester, West Sussex, South East England, England, U.K., 2008.
- [199] Allan Kelly. Xanpan: Team Centric Agile Software Development – Combining Kanban and XP - inspiration for creating your own hybrid. Software Strategy Ltd., London, U.K., 2015.
- [200] Joshua Kerievsky. Refactoring to Patterns. Addison-Wesley Signature Series. Addison-Wesley, Boston, MA, 2005.
- [201] Brian W. Kernighan and Rob Pike. The Practice of Programming. Addison-Wesley Professional Computing Series. Addison-Wesley, Reading, MA, 1999.
- [202] Brian W. Kernighan and P. J. Plauger. The Elements of Programming Style. McGraw-Hill, New York, NY, second edition, 1982.
- [203] Kurt Keutzer, Sharad Malik, A. Richard Newton, Jan M. Rabaey, and Alberto Sangiovanni-Vincentelli. System-level design: Orthogonalization of concerns and platform-based design. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 19(12):1523–1543, December 2000.
- [204] Joshua Kim. “whiplash”, the MIT media lab, and the rest of us. Available online from *Inside Higher Ed: Higher Education Blogs: Blog U* at: <https://www.insidehighered.com/blogs/technology-and-learning/whiplash-mit-media-lab-and-rest-us>; February 24, 2017 was the last accessed date, January 22 2017.
- [205] Donald E. Knuth. Literate programming. The Computer Journal, 27(2):97–111, 1984.
- [206] Donald E. Knuth. Literate Programming. Center for the Study of Language and Information – Lecture Notes. The University of Chicago Press, Chicago, IL, 1992.
- [207] Philip Koopman. Better Embedded System Software. Drumnadrochit Education, 2010.
- [208] Helmut Kopka and Patrick W. Daly. Guide to L^AT_EX. Addison-Wesley Series on Tools and Techniques for Computer Typesetting. Addison-Wesley, Boston, MA, fourth edition, 2004.

- [209] Derrick G. Kourie and Bruce W. Watson. The Correctness-by-Construction Approach to Programming. Springer-Verlag Berlin Heidelberg, Heidelberg, Germany, 2012.
- [210] R. S. Krishnamachari and P. Y. Papalambros. Hierarchical decomposition synthesis in optimal systems design. ASME Journal of Mechanical Design, 119(4):448–457, December 1 1997.
- [211] Ramprasad S. Krishnamachari and Panos Y. Papalambros. Hierarchical decomposition synthesis in optimal systems design. Technical Report UMR2480, University of Michigan, Ann Arbor, MI, August 1995.
- [212] Shriram Krishnamurthi. Artifact evaluation for software conferences. Available online from *Brown University: Computer Science Department: Prof. Shriram Krishnamurthi's Web page* at: <http://www.artifact-eval.org>; self-published; June 25, 2017 was the last accessed date.
- [213] Bruce H. Krogh, Edward Lee, Insup Lee, Al Mok, George Pappas, Raj Rajkumar, Harvey Rubin, Alberto Sangiovanni-Vincentelli, Lui Sha, Kang Shin, Jack Stankovic, Janos Sztipanovits, Wayne Wolf, and Wei Zhao. Report: Cyber-physical systems summit. Technical report, National Science Foundation, St. Louis, MO, April 24–25 2008.
- [214] Thomas Kropf. Introduction to Formal Hardware Verification. Springer-Verlag Berlin Heidelberg, Heidelberg, Germany, 1999.
- [215] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. Technical debt: From metaphor to theory and practice. IEEE Software, 29(6):18–21, November–December 2012.
- [216] Vijay Kumar. 101 Design Methods: A Structured Approach for Driving Innovation in Your Organization. John Wiley & Sons, Hoboken, NJ, 2013.
- [217] Sudipta Kundu, Sorin Lerner, and Rajesh K. Gupta. High-Level Verification: Methods and Tools for Verification of System-Level Designs. Springer Science+Business Media, LLC, New York, NY, 2011.
- [218] David Kushner. Facebook philosophy: Move fast and break things – hacker culture is alive and well at Facebook. Available online from *IEEE Spectrum* at: <http://spectrum.ieee.org/at-work/innovation/facebook-philosophy-move-fast-and-break-things/0>; October 9, 2014 was the last accessed date, June 1 2011.
- [219] Gayle Laakmann. Cracking the technical interview: 150 technical interview questions and solutions, written by experts. Self-published, 2009.
- [220] Jean J. Labrosse. Embedded Systems Building Blocks: Complete and Ready-to-Use Modules in C. CMP Books, Lawrence, KS, second edition, 1999.
- [221] Sylvia Lafair. Don't Bring It to Work: Breaking the Family Patterns that Limit Success. Jossey-Bass, San Francisco, CA, 2009.
- [222] Phillip A. Laplante. Real-Time Systems Design and Analysis. John Wiley & Sons, Hoboken, NJ, third edition, 2004.
- [223] Phillip A. Laplante. Requirements Engineering for Software and Systems. Auerbach Applied Software Engineering. CRC Press, Boca Raton, FL, second edition, 2014.

- [224] Phillip A. Laplante and Seppo J. Ovaska. Real-Time Systems Design and Analysis: Tools for the Practitioner. John Wiley & Sons and IEEE Press, Hoboken, NJ and Piscataway, NJ, fourth edition, 2012.
- [225] Craig Larman. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process. Prentice-Hall, Upper Saddle River, NJ, second edition, 2002.
- [226] Craig Larman. Agile and Iterative Development: A Manager’s Guide. The Agile Software Development. Addison-Wesley, Boston, MA, 2004.
- [227] Craig Larman. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. Prentice Hall PTR, Upper Saddle River, NJ, third edition, 2005.
- [228] Craig Larman and Victor R. Basili. Iterative and incremental development: A brief history. IEEE Computer, 36(6):47–56, June 2003.
- [229] Craig Larman and Bas Vodde. Scaling Lean & Agile Development: Thinking and Organizational Tools for Large-Scale Scrum. Pearson Education, Boston, MA, 2009.
- [230] Craig Larman and Bas Vodde. Practices for Scaling Lean & Agile Development: Large, Multisite, and Offshore Product Development with Large-Scale Scrum. Pearson Education, Boston, MA, 2010.
- [231] Diana Larsen and Ainsley Nies. Liftoff: Launching agile teams & projects. Self-published, Hillsboro, OR, 2011.
- [232] Edward Ashford Lee and Sanjit Arunkumar Seshia. Introduction to Embedded Systems: A Cyber-Physical Systems Approach. The MIT Press, Cambridge, MA, second edition, 2017.
- [233] Kent D. Lee. Programming Languages: An Active Learning Approach. Springer Science+Business Media, LLC, New York, NY, 2008.
- [234] Dean Leffingwell. Agile Software Requirements: Lean Requirements Practices for Teams, Programs, and the Enterprise. Agile Software Development Series. Addison-Wesley, Boston, MA, 2011.
- [235] Mike Levin. Incurring more personal technical debt, but so what? Available online from *Medium: Mike Levin, SEO in NYC* at: <https://medium.com/@mikelevinseo/incurring-more-personal-technical-debt-but-so-what-aad4bca1fa5c>; September 27, 2018 was the last accessed date, October 27 2017.
- [236] Ming Li and Paul Vitányi. An Introduction to Kolmogorov Complexity and Its Applications. Texts in Computer Science. Springer Science+Business Media, LLC, New York, NY, third edition, 2008.
- [237] Mark Liberman. Replicability vs. reproducibility – or is it the other way around? Available online from *University of Pennsylvania: School of Arts & Sciences: Linguistic Data Consortium (LDC)* at: <http://languagelog.ldc.upenn.edu/n11/?p=21956>; April 21, 2017 was the last accessed date, October 31 2015.

- [238] Eric Lippert. Vexing exceptions. Available online from *Microsoft Corporation: Microsoft Developer network: MSDN Blogs: Fabulous Adventures In Coding, Eric Lippert's Erstwhile Blog* at: <https://blogs.msdn.microsoft.com/ericlippert/2008/09/10/vexing-exceptions/>; September 19, 2018 was the last accessed date, September 10 2008.
- [239] Fred Long, Dhruv Mohindra, Robert C. Seacord, Dean F. Sutherland, and David Svoboda. Java™ Coding Guidelines: 75 Recommendations for Reliable and Secure Programs. The SEI Series in Software Engineering. Addison-Wesley, Upper Saddle River, NJ, 1995.
- [240] Kenneth C. Loudon and Kenneth Alfred Lambert. Programming Languages: Principles and Practices. Course Technology, Boston, MA, 2012.
- [241] Zhonghai Lu. Design and Analysis of On-Chip Communication for Network-on-Chip Platforms. PhD thesis, KTH Royal Institute of Technology, Stockholm, Sweden, March 2007.
- [242] Ian Macdonald. The unofficial Ruby usage guide. Available online from *Caliban – Opinion and Righteous Anger: Ruby Projects: Recommended Tutorials* at: <http://www.caliban.org/ruby/rubyguide.shtml>; self-published; October 20, 2015 was the last accessed date.
- [243] Vijay K. Madisetti and Chonlameth Arpikanondt. A Platform-Centric Approach to System-on-Chip (SOC) Design. Springer Science+Business Media, Inc., Boston, MA, 2005.
- [244] Thomas Mailund. Beginning Data Science in R: Data Analysis, Visualization, and Modelling for the Data Scientist. Apress Media, LLC, Berkeley, CA, 2017.
- [245] Gary Marrer. Fundamentals of Programming: With Object Orientated Programming. Laptop Press LLC, python edition, November 30 2009.
- [246] Grant Martin and Henry Chang. Winning the SoC Revolution: Experiences in Real Design. Kluwer Academic Publishers, Norwell, MA, 2003.
- [247] Robert C. Martin. Clean Code: A Handbook of Agile Software Craftsmanship. Robert C. Martin. Pearson Education, Upper Saddle River, NJ, 2009.
- [248] Robert C. Martin and Micah Martin. Agile Principles, Patterns, and Practices in C#. Robert C. Martin. Prentice Hall, Boston, MA, 2007.
- [249] Robert Cecil Martin, James W. Newkirk, and Robert S. Koss. Agile Software Development: Principles, Patterns, and Practices. Alan Apt. Prentice-Hall, Upper Saddle River, NJ, 2002.
- [250] Jon Masters and Richard Blum. Professional Linux Programming. Programmer to Programmer™. Wiley Publishing, Indianapolis, IN, 2007.
- [251] John C. Maxwell. Everyone Communicates, Few Connect: What the Most Effective People Do Differently. Thomas Nelson, Inc., Nashville, TN, 2010.
- [252] Steve McConnell. Rapid Development: Taming Wild Software Schedules. Microsoft Press, Redmond, WA, 1996.
- [253] Steve McConnell. Code Complete: A Practical Handbook of Software Construction. Microsoft Press, Redmond, WA, second edition, 2004.
- [254] Patty McCord. Powerful: Building a Culture of Freedom and Responsibility. Silicon Guild, 2017.

- [255] Annabelle McIver and Carroll Morgan. Programming Methodology. Monographs in Computer Science. Springer-Verlag New York, New York, NY, 2003.
- [256] Annabelle McIver and Carroll Morgan. Abstraction, Refinement and Proof for Probabilistic Systems. Monographs in Computer Science. Springer Science+Business Media, Inc., New York, NY, 2005.
- [257] Paul McLellan. Agile IC development. Available online from *SemiWiki.com: Semiconductor Expert Forum* at: <https://www.semiwiki.com/forum/content/3889-agile-ic-development.html>; September 23, 2018 was the last accessed date, October 1 2014.
- [258] Mendeley Limited staff. Harvard format citation guide. Available online from *Welcome to Mendeley: Support: Help Guides: Guides: Help guides: Citation Guides: Citation Hub: Harvard* at: <https://www.mendeley.com/guides/harvard-citation-guide/>; March 7, 2023 was the last accessed date, 2022.
- [259] Mendeley Limited staff. Ultimate citation cheat sheet. Available online from *Welcome to Mendeley: Support: Help Guides: Guides: Help guides: Citation Guides: Citation Hub: Contents: Ultimate Citation Cheat Sheet* at: <https://www.mendeley.com/guides/ultimate-citation-cheat-sheet/>; March 7, 2023 was the last accessed date, 2022.
- [260] Mercurial contributors. Learning Mercurial in workflows. Available online from *Mercurial community: Mercurial SCM* at: <https://www.mercurial-scm.org/guide>; self-published; February 15, 2019 was the last accessed date.
- [261] Mercurial contributors. Mercurial SCM. Available online from *Mercurial community* at: <https://www.mercurial-scm.org/>; self-published; February 15, 2019 was the last accessed date.
- [262] Mercurial contributors. A tutorial on using Mercurial. Available online from *Mercurial community: Mercurial SCM: Mercurial Wiki*
at: <https://www.mercurial-scm.org/wiki/Tutorial>; self-published; February 15, 2019 was the last accessed date, September 2 2013.
- [263] Mercurial contributors. Understanding mercurial. Available online from *Mercurial community: Mercurial SCM: Mercurial Wiki*
at: <https://www.mercurial-scm.org/wiki/UnderstandingMercurial>; self-published; February 15, 2019 was the last accessed date, September 2 2013.
- [264] Mercurial contributors. Mercurial
wiki
. Available online from *Mercurial community: Mercurial SCM* at: <https://www.mercurial-scm.org/wiki/>; self-published; February 15, 2019 was the last accessed date, February 1 2019.
- [265] Gerard Meszaros. xUnit Test Patterns: Refactoring Test Code. Addison-Wesley Signature Series. Pearson Education, Boston, MA, 2007.
- [266] Bertrand Meyer. Object-Oriented Software Construction. Prentice Hall PTR, Upper Saddle River, NJ, second edition, 1997.

- [267] Ali Mili and Fairouz Tchier. Software Testing: Concepts and Operations. John Wiley & Sons, Hoboken, NJ, 2015.
- [268] Mike Mintz and Robert Ekendahl. Hardware Verification with *System Verilog*: An Object-Oriented Framework. Springer Science+Business Media, LLC, New York, NY, 2007.
- [269] Trevor Misfeldt, Gregory Bumgardner, and Andrew Gray. The Elements of C++ Style. Cambridge University Press, New York, NY, 2004.
- [270] Jayadev Misra. A Discipline of Multiprogramming: Programming Theory for Distributed Applications. Monographs in Computer Science. Springer-Verlag New York, New York, NY, 2001.
- [271] John C. Mitchell. Concepts in Programming Languages. Cambridge University Press, Cambridge, U.K., 2002.
- [272] Frank Mittelbach, Michel Goossens, Johannes Braams, David Carlisle, and Chris Rowley. The L^AT_EX Companion. Addison-Wesley Series on Tools and Techniques for Computer Typesetting. Addison-Wesley, Boston, MA, second edition, 2004.
- [273] Vince Molinaro. The Leadership Contract: The Fine Print to Becoming an Accountable Leader. John Wiley & Sons, Hoboken, NJ, second edition, 2016.
- [274] Andy Molinsky. Reach: A New Strategy to Help You Step Outside Your Comfort Zone, Rise to the Challenge, and Build Confidence. Avery, New York, NY, 2017.
- [275] Antonio F. Mondragon-Torres. An agile embedded systems capstone course: Overview, experiences, and lessons learned. In Proceedings of the 43rd IEEE Frontiers in Education Conference (FIE 2013), Oklahoma City, OK, October 23–26 2013. American Society for Engineering Education, Institute of Electrical and Electronics Engineers, and IEEE Computer Society, IEEE Press.
- [276] Antonio F. Mondragon-Torres, Alexander Kozitsky, Clifford Bundick, Edward Mc Kenna Jr., Eric Alley, Matthew Lloyd, Peter Stanley, and Roger Lane. Work in progress – an agile embedded systems design capstone course. In Proceedings of the 41st Annual ASEE/IEEE Frontiers in Education Conference (FIE 2011), pages F4F–1 – F4F–3, Rapid City, SD, October 12–15 2011. American Society for Engineering Education, Institute of Electrical and Electronics Engineers, and IEEE Computer Society, IEEE Press.
- [277] Jean-François Monin and Michael G. Hinchey. Understanding Formal Methods. Springer-Verlag London, London, U.K., 2003.
- [278] Daniel J. Mosley and Bruce A. Posey. Just Enough Software Test Automation. Just Enough, Yourdon Press Series. Prentice Hall, Upper Saddle River, NJ, 2002.
- [279] Matthias Müller-Hannemann and Stefan Schirra. Algorithm Engineering: Bridging the Gap between Algorithm Theory and Practice, volume 5971 of Lecture Notes in Computer Science. Springer-Verlag Berlin Heidelberg, Heidelberg, Germany, 2010.
- [280] Ware Myers. Why software developers refuse to improve. IEEE Computer, 31(4):110–112, April 1998.

- [281] Mythic Inc. staff. Hardware DevOps engineer. Available online from *LinkedIn: Jobs: Search All Jobs* at: <https://www.linkedin.com/jobs/view/hardware-devops-engineer-at-mythic-1078283501>; September 10, 2019 was the last accessed date, December 2018.
- [282] Huy Nguyen, Hai-My Nguyen, and Minh-Luan Tran. *TexPaste: Pastebin for mathematicians*. Available online at: <http://www.texpaste.com>; self-published; June 4, 2018 was the last accessed date, October 12 2013.
- [283] Pierluigi Nuzzo and Alberto Sangiovanni-Vincentelli. Robustness in analog systems: Design techniques, methodologies and tools. In *Proceedings of the 6th IEEE International Symposium on Industrial Embedded Systems (SIES 2011)*, pages 194–203, Västerås, Sweden, June 15–17 2011. IEEE Press.
- [284] Dan Olsen. *The Lean Product Playbook: How to Innovate with Minimum Viable Products and Rapid Customer Feedback*. John Wiley & Sons, Hoboken, NJ, 2015.
- [285] Ijeoma Oluo. *So You Want to Talk About Race*. Seal Press, New York, NY, 2019.
- [286] Zhiyang Ong. Test automation for software quality assurance: A comparative study of hardware and software test automation. Available online from *Google: Google Sites: Zhiyang Ong's Web page* at: https://sites.google.com/site/zhiyangong/zhiyang_internship_report2.pdf?attredirects=0; March 3, 2011 was the last accessed date, August 8 2006.
- [287] Zhiyang Ong. *bibtex-analytics*. Available online from *GitHub: Zhiyang Ong's GitHub Profile* at: <https://github.com/eda-ricercatore/bibtex-analytics>; April 11, 2023 was the last accessed date, April 11 2023.
- [288] Oracle Corporation staff. How to write Doc comments for the Javadoc tool. Available online from *Oracle Corporation: Oracle Technology Network: Recently Published Technical Articles: Articles About Java Technology* at: <http://www.oracle.com/technetwork/articles/java/index-137868.html>; January 30, 2017 was the last accessed date.
- [289] Oracle Corporation staff. Javadoc tool. Available online from *Oracle Corporation: Oracle Technology Network: Recently Published Technical Articles: Articles About Java Technology* at: <http://www.oracle.com/technetwork/articles/java/index-jsp-135444.html>; January 30, 2017 was the last accessed date.
- [290] Oracle Corporation staff. Javadoc tool. Available online from *Oracle Corporation: Oracle Technology Network: Oracle Technology Network for Java Developers: Java SE at a Glance: Java SE Documentation* at: <http://www.oracle.com/technetwork/java/javase/documentation/index-jsp-135444.html>; January 30, 2017 was the last accessed date.
- [291] Oracle Corporation staff. javadoc. Available online from *Oracle Corporation: Oracle Help Center: Java Platform, Standard Edition (Java SE) 8: Java Platform Standard Edition 8 Documentation: JDK Tools and Utilities: Java Platform, Standard Edition Tools Reference: §5 Create and Build Applications* at: <https://docs.oracle.com/javase/8/docs/technotes/tools/windows/javadoc.html>; January 30, 2017 was the last accessed date, January 2016.
- [292] Andrew Oram and Steve Talbott. *Managing Projects with make: C Programming Utility*. O'Reilly & Associates, Sebastopol, CA, second edition, 1991.

- [293] Andy Oram and Greg Wilson. Beautiful Code: Leading Programmers Explain How They Think. O'Reilly Media, Sebastopol, CA, 2007.
- [294] Andy Oram and Greg Wilson. Making Software: What Really Works, and Why We Believe It. Theory in Practice. O'Reilly Media, Sebastopol, CA, 2011.
- [295] Ted Osborne and Salas leVirve. Google Python style guide. Available online from *GitHub: Google: styleguide* at: <http://google.github.io/styleguide/pyguide.html> and <https://github.com/google/styleguide/blob/gh-pages/pyguide.md>; September 17, 2019 was the last accessed date, December 5 2018.
- [296] Ted Osborne and Nodir Turakulov. Philosophy. Available online from *GitHub: Google: styleguide: Google documentation guide* at: <https://github.com/google/styleguide/blob/gh-pages/docguide/philosophy.md>; September 17, 2019 was the last accessed date, January 30 2018.
- [297] Ted Osborne, Nodir Turakulov, and Ackermann Yuriy. Documentation best practices. Available online from *GitHub: Google: styleguide: Google documentation guide* at: https://github.com/google/styleguide/blob/gh-pages/docguide/best_practices.md; September 17, 2019 was the last accessed date, January 30 2018.
- [298] Bryan O'Sullivan. Mercurial: The Definitive Guide. Modern Software for Collaboration. O'Reilly Media, Sebastopol, CA, 2009.
- [299] Harry J. Paarsch and Konstantin Golyaev. A Gentle Introduction to Effective Computing in Quantitative Research: What Every Research Assistant Should Know. The MIT Press, London, U.K., 2016.
- [300] Ashwin Pajankar. Python Unit Test Automation: Practical Techniques for Python Developers and Testers. Apress Media, LLC, Berkeley, CA, 2017.
- [301] Rachel A. Parker and Richard P. Appelbaum. Can Emerging Technologies Make a Difference in Development? Routledge, New York, NY, 2012.
- [302] Partnership on AI staff. The partnership on ai to benefit people and society. Available online at: <https://www.partnershiponai.org/>; March 27, 2021 was the last accessed date, 2020.
- [303] Oren Patashnik. BIBTEXing. Available online from *Comprehensive T_EX Archive Network: tex-archive: biblio: bibtex: base* at: <https://ctan.org/tex-archive/biblio/bibtex/base> and <http://mirrors.ctan.org/biblio/bibtex/base/btxdoc.pdf>; September 24, 2014 was the last accessed date, February 8 1988.
- [304] Carlo Pescio. Principles versus patterns. IEEE Computer, 30(9):130–131, September 1997.
- [305] Christopher Peterson and Martin E. P. Seligman. Character Strengths and Virtues: A Handbook and Classification. Oxford University Press, New York, NY, 2004.
- [306] Benjamin C. Pierce. Types and Programming Languages. The MIT Press, Cambridge, MA, 2002.
- [307] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Brent Yorgey, Loris D'Antoni, Andrew W. Appel, Arthur Chargueraud, Anthony Cowley, Jeffrey Foster, Dmitri Garbuzov, Michael Hicks, Ranjit Jhala, Greg Morrisett, Jennifer Paykin, Mukund Raghothaman, Chung-chieh Shan, Leonid Spesivtsev, Andrew Tolmach, Stephanie Weirich, and Steve Zdancewic. Software Foundations. University of Pennsylvania, Philadelphia, PA, Spring 2017.

- [308] Hasso Plattner, Christoph Meinel, and Larry Leifer. Design Thinking: Understand – Improve – Apply. Understanding Innovation. Springer-Verlag Berlin Heidelberg, Heidelberg, Germany, 2011.
- [309] Zissis Poulos and Andreas Veneris. Failure triage in RTL regression verification. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 37(9):1893–1906, September 2018.
- [310] Zissis Poulos, Yu-Shen Yang, and Andreas Veneris. A failure triage engine based on error trace signature extraction. In Proceedings of the 15th IEEE Symposium On-Line Testing (IOLTS 2013), pages 73–78, Chania, Crete, Greece, July 8–10 2013. IEEE Council on EDA (CEDA), and IEEE Computer Society Test Technology Technical Council (TTTC), IEEE Press.
- [311] Zissis Poulos, Yu-Shen Yang, Andreas Veneris, and Bao Le. Simulation and satisfiability guided counter-example triage for RTL design debugging. In Proceedings of the 15th IEEE International Symposium on Quality Electronic Design (ISQED 2014), pages 618–624, Santa Clara, CA, March 3–5 2014. IEEE Electron Devices Society (EDS), IEEE Reliability Society, IEEE Components, Packaging and Manufacturing Technology Society, and IEEE Circuits and Systems Society (CAS), IEEE Press.
- [312] Zissis Paraskevas Poulos. High level debugging techniques for modern verification flows. Master’s thesis, University of Toronto, Toronto, Ontario, Canada, July 4 2014.
- [313] Zissis Paraskevas Poulos. Scalable Inference in Hardware Verification and Social Graph Reasoning. PhD thesis, University of Toronto, Toronto, Ontario, Canada, November 2018.
- [314] Terrence W. Pratt and Marvin V. Zelkowitz. Programming Languages: Design and Implementation. Prentice Hall, Upper Saddle River, NJ, fourth edition, 2001.
- [315] Christian R. Prause. Reputation-based self-management of software process artifact quality in consortium research projects. In Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE’11), pages 380–383, Szeged, Hungary, September 5–9 2011. ACM Special Interest Group on Software Engineering (SIGSOFT), ACM Press.
- [316] Jeff Preshing. C++ has become more Pythonic. Available online from *Preshing on Programming* at: <http://preshing.com/20141202/cpp-has-become-more-pythonic/>; February 1, 2018 was the last accessed date, December 2 2014.
- [317] Roger S. Pressman. Software Engineering: A Practitioner’s Approach. McGraw-Hill Series in Computer Science. McGraw-Hill, New York, NY, fifth edition, 2001.
- [318] Roger S. Pressman. Software Engineering: A Practitioner’s Approach. McGraw-Hill, New York, NY, seventh edition, 2010.
- [319] Programming Research Ltd. staff. High integrity C++: Coding standard version 4.0. Technical report, Programming Research Ltd., Hersham, Surrey, South East England, England, U.K., October 3 2013.
- [320] Ken Pugh. Lean-Agile Acceptance Test-Driven Development: Better Software Through Collaboration. Net Objectives Lean-Agile. Addison-Wesley, Boston, MA, 2011.
- [321] Robert D. Putnam. Bowling Alone: The Collapse and Revival of American Community. Simon & Schuster, New York, NY, 2000.

- [322] Ben Rady and Rod Coffin. Continuous Testing: With Ruby, Rails, and JavaScript. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, 2011.
- [323] Prakash Rashinkar, Peter Paterson, and Leena Singh. System-on-a-Chip Verification: Methodology and Techniques. Kluwer Academic Publishers, Dordrecht, The Netherlands, 2002.
- [324] Jonathan Rasmusson. The Agile Samurai: How Agile Masters Deliver Great Software. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, September 15 2010.
- [325] Eric S. Raymond. The Art of UNIX Programming. Addison-Wesley Professional Computing Series. Pearson Education, Boston, MA, 2004.
- [326] Robert L. Read. How to be a programmer: A short, comprehensive, and personal summary. Available online from *Colorado School of Mines: Samizdat Press: Free stuff on this site: How to be a Programmer: How to be a Programmer – A Short, Comprehensive, and Personal Summary* at: <http://samizdat.mines.edu/howto/HowToBeAProgrammer.html> and <http://samizdat.mines.edu/howto/HowToBeAProgrammer.pdf>; April 2, 2014 was the last accessed date, 2003.
- [327] Martin Reddy. API Design for C++. Morgan Kaufmann, Burlington, MA, 2011.
- [328] Martin Rees. Time for global statistics we can count on. Nature, 502(7471):272, October 17 2013.
- [329] Andrew Reiner. Teaching men to be emotionally honest. Available online from *The New York Times: Education: Education Life: Notebook* at: http://www.nytimes.com/2016/04/10/education/edlife/teaching-men-to-be-emotionally-honest.html?_r=0; September 6, 2016 was the last accessed date, April 4 2016.
- [330] Kenneth Reitz. Code style. Available online from *The Hitchhiker’s Guide to Python* at: <http://docs.python-guide.org/en/latest/writing/style/>; self-published; February 1, 2018 was the last accessed date, 2016.
- [331] Kenneth Reitz and Tanya Schlusser. The Hitchhiker’s Guide to Python: Best Practices for Development. O’Reilly Media, Sebastopol, CA, 2016.
- [332] John C. Reynolds. Theories of Programming Languages. Cambridge University Press, New York, NY, 1998.
- [333] Don Rheem. Thrive By Design: The Neuroscience That Drives High-Performance Cultures. ForbesBooks, Charleston, SC, 2017.
- [334] Camille Gear Rich and Renée Smith-Maddox. 2021 DEI week session repository. Available online from *University of Southern California: Diversity, Equity, and Inclusion Week* at: <https://deiweek.usc.edu/2021-dei-week-session-repository/>; April 24, 2022 was the last accessed date, 2022.
- [335] Leanna Rierson. Developing Safety-Critical Software: A Practical Guide for Aviation Software and DO-178C Compliance. CRC Press, Boca Raton, FL, 2013.
- [336] Chris Riesbeck. Useful C++ / Unix resources. Available online from *Prof. Chris Riesbeck’s Web page: Programming*, Computer Science Division, Department of Electrical Engineering and Computer Science, Robert R. McCormick School of Engineering and Applied Science, Northwestern University at: <http://www.cs.northwestern.edu/~riesbeck/programming/c++/>; September 30, 2015 was the last accessed date, July 2 2009.

- [337] Suzanne Robertson and James Robertson. Mastering the Requirements Process: Getting Requirements Right. Addison-Wesley, Upper Saddle River, NJ, third edition, 2013.
- [338] Kristian Rother. Pro Python Best Practices: Debugging, Testing and Maintenance. Apress Media, LLC, Berkeley, CA, 2017.
- [339] Johanna Rothman. Manage It!: Your Guide to Modern, Pragmatic Project Management. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, 2007.
- [340] Johanna Rothman. Manage Your Project Portfolio: Increase Your Capacity and Finish More Projects. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, 2009.
- [341] Johanna Rothman. Manage Your Project Portfolio: Increase Your Capacity and Finish More Projects. The Pragmatic Programmers, Raleigh, NC, second edition, 2016.
- [342] Johanna Rothman. Create Your Successful Agile Project: Collaborate, Measure, Estimate, Deliver. The Pragmatic Programmers, Raleigh, NC, 2017.
- [343] RunMyCode Association contributors. RunMyCode. Available online at: <http://www.runmycode.org/>; November 13, 2016 was the last accessed date, 2013.
- [344] Pardis Sabeti and Lara Salahi. Outbreak Culture: The Ebola Crisis and the Next Epidemic. Harvard University Press, Cambridge, MA, 2018.
- [345] Sean A. Safarpour. Formal Methods in Automated Design Debugging. PhD thesis, University of Toronto, Toronto, Ontario, Canada, 2009.
- [346] David Sale. Testing Python: Applying Unit Testing, TDD, BDD and Acceptance Testing. John Wiley & Sons, Chichester, West Sussex, South East England, England, U.K., 2014.
- [347] Resve Saleh, Steve Wilton, Shahriar Mirabbasi, Alan Hu, Mark Greenstreet, Guy Lemieux, Partha Pratim Pande, Cristian Grecu, and Andre Ivanov. System-on-chip: Reuse and integration. Proceedings of the IEEE, 94(6):1050–1069, June 2006.
- [348] Miro Samek. Practical UML Statecharts in C/C++: Event-Driven Programming for Embedded Systems. Newnes, Burlington, MA, second edition, 2009.
- [349] Sheryl Sandberg and Nell Scovell. Lean In: Women, Work, and the Will to Lead. Alfred A. Knopf, New York, NY, 2013.
- [350] Sheryl Sandberg and Nell Scovell. Lean In for Graduates: With New Chapters by Experts, Including Find Your First Job, Negotiate Your Salary, and Own Who You Are. Alfred A. Knopf, New York, NY, 2014.
- [351] Alberto Sangiovanni-Vincentelli. Quo vadis, SLD? reasoning about the trends and challenges of system level design. Proceedings of the IEEE, 95(3):467–506, March 2007.
- [352] Alberto Sangiovanni-Vincentelli. Is a unified methodology for system-level design possible? IEEE Design & Test of Computers, 25(4):346–357, July–August 2008.
- [353] Alberto Sangiovanni-Vincentelli and Grant Martin. Platform-based design and software design methodology for embedded systems. IEEE Design & Test of Computers, 18(6):23–33, November–December 2001.

- [354] Sergio Saponara, Pierluigi Nuzzo, Paolo D'Abramo, and Luca Fanucci. EURASIP Journal on Embedded Systems: Design Methodologies and Innovative Architectures for Mixed-Signal Embedded Systems. Hindawi Publishing Corporation, Cairo, Egypt, 2010.
- [355] Peter Savage. Git in the trenches. Available online from *GitHub: Peter Savage* at: <http://cbx33.github.io/gitt/>, <http://cbx33.github.io/gitt/download.html>, and <https://github.com/cbx33/gitt>; self-published; August 1, 2014 was the last accessed date, August 2011.
- [356] Stephen R. Schach. Object-Oriented and Classical Software Engineering. McGraw-Hill, New York, NY, fifth edition, 2002.
- [357] Stephen R. Schach. Object-Oriented and Classical Software Engineering. McGraw-Hill, New York, NY, seventh edition, 2007.
- [358] Stephen R. Schach. Object-Oriented and Classical Software Engineering. McGraw-Hill, New York, NY, eighth edition, 2011.
- [359] Quirin Schiermeier. Data management made simple. Nature, 555(7697):403–405, March 13 2018.
- [360] Anna Schneider. How to become a data scientist before you graduate. Berkeley Science Review, July 30 2013.
- [361] Helmut Schwichtenberg and Ralf Steinbrüggen, editors. Proceedings of the NATO Advanced Science Institute on Proof and System-Reliability, volume 62 of NATO Science Series II: Mathematics, Physics and Chemistry, Marktoberdorf, Germany, July 24 – August 5 2001. Kluwer Academic Publishers.
- [362] Science Infusion Software Engineering Process Group staff. Science infusion software engineering process group (SISEPG) C++ programming standards and guidelines. Available online from *United States Department of Commerce: National Oceanic and Atmospheric Administration: National Weather Service: Office of Hydrologic Development, now known as the National Water Center: Hydrology Laboratory* as Version 1.11 at: http://www.nws.noaa.gov/oh/hrl/developers_docs/C++_Software_Standards.pdf; October 27, 2015 was the last accessed date, November 17 2006.
- [363] Michael L. Scott. Programming Language Pragmatics. Morgan Kaufmann, Burlington, MA, third edition, 2009.
- [364] Peter Sestoft. Programming Language Concepts, volume 50 of Undergraduate Topics in Computer Science. Springer-Verlag London, London, U.K., 2012.
- [365] Mary Shapiro. HBR Guide to Leading Teams: Balance skills and styles. Establish clear roles. Promote healthy dissent. Harvard Business Review Guides. Harvard Business Review Press, Boston, MA, 2015.
- [366] James Shore and Shane Warden. The Art of Agile Development. Theory in Practice. O'Reilly Media, Sebastopol, CA, 2008.
- [367] Joseph Sifakis. Rigorous system design. Foundations and Trends® in Electronic Design Automation, 6(4):293–362, April 15 2013.
- [368] Leena Singh, Leonard Drucker, and Neyaz Khan. Advanced Verification Techniques: A SystemC Based Approach for Successful Tapeout. Kluwer Academic Publishers, Boston, MA, 2004.

- [369] Kevin A. Smith and Doug Kramer. Requirements for writing Java API specifications. Available online from *Oracle Corporation: Oracle Technology Network: Oracle Technology Network for Java Developers: Java SE at a Glance: Java SE Documentation* at: <http://www.oracle.com/technetwork/java/javase/documentation/index-142372.html>; January 30, 2017 was the last accessed date, January 2003.
- [370] Ian Sommerville. *Software Engineering*. Pearson Education, Harlow, Essex, East of England, England, U.K., eighth edition, 2007.
- [371] Joel Spolsky. *Joel on Software: And on Diverse and Occasionally Related Matters That Will Prove of Interest to Software Developers, Designers, and Managers, and to Those Who, Whether by Good Fortune or Ill Luck, Work with Them in Some Capacity*. Apress, Berkeley, CA, 2004.
- [372] Joel Spolsky. *More Joel on Software: Further Thoughts on Diverse and Occasionally Related Matters That Will Prove of Interest to Software Developers, Designers, and Managers, and to Those who, Whether by Good Fortune or Ill Luck, Work with Them in Some Capacity*. Apress, Berkeley, CA, 2008.
- [373] Richard Stallman and Bob Chassell. GNU Texinfo 6.3. Available online from *The GNU Operating System and the Free Software Movement: GNU Software: Texinfo – The GNU Documentation System: Texinfo manuals: GNU Texinfo manual* at: <https://www.gnu.org/software/texinfo/manual/texinfo/texinfo.html>; November 4, 2016 was the last accessed date, September 11 2016.
- [374] Richard Stallman and Bob Chassell. GNU Texinfo manual. Available online from *The GNU Operating System and the Free Software Movement: GNU Software: Texinfo – The GNU Documentation System: Texinfo manuals* at: <https://www.gnu.org/software/texinfo/manual/texinfo/>; November 4, 2016 was the last accessed date, September 11 2016.
- [375] Richard Stallman and Bob Chassell. Texinfo: The GNU documentation system. Available online from *The GNU Operating System and the Free Software Movement: GNU Software* at: <https://www.gnu.org/software/texinfo/>; November 4, 2016 was the last accessed date, September 11 2016.
- [376] Mircea R. Stan, Paul D. Franzon, Seth Copen Goldstein, John C. Lach, and Matthew M. Ziegler. Molecular electronics: From devices and interconnect to circuits and architecture. *Proceedings of the IEEE*, 91(11):1940–1957, November 2003.
- [377] Andrew Stellman and Jennifer Greene. *Learning Agile: Understanding Scrum, XP, Lean, and Kanban*. O'Reilly Media, Sebastopol, CA, 2015.
- [378] Dale J. Stephens. *Hacking Your Education: Ditch the Lectures, Save Tens of Thousands, and Learn More Than Your Peers Ever Will*. Penguin Group, New York, NY, 2013.
- [379] Chris Sterling. *Managing Software Debt: Building for Inevitable Change*. The Agile Software Development. Addison-Wesley, Boston, MA, 2011.
- [380] Perdita Stevens and Rob Pooley. *Using UML: Software Engineering with Objects and Components*. Addison-Wesley Object Technology Series. Addison-Wesley, Harlow, Essex, East of England, England, U.K., updated edition, 2000.

- [381] Thomas Stober and Uwe Hansmann. Agile Software Development: Best Practices for Large Software Development Projects. Springer-Verlag Berlin Heidelberg, Heidelberg, Germany, 2010.
- [382] Victoria Stodden, Friedrich Leisch, and Roger D. Peng. Implementing Reproducible Research. Chapman & Hall/CRC The R Series. CRC Press, Boca Raton, FL, 2014.
- [383] John R. Stoker. Overcoming Fake Talk: How to Hold REAL Conversations that Create Respect, Build Relationships, and Get Results. McGraw-Hill Education, New York, NY, 2013.
- [384] Venkat Subramaniam and Andy Hunt. Practices of an Agile Developer: Working in the Real World. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, 2006.
- [385] Joshua Suereth and Matthew Farwell. sbt in Action: The simple Scala build tool. Manning Publications Co., Shelter Island, NY, 2016.
- [386] Xuening Sun. Compositional Design of Analog Systems Using Contracts. PhD thesis, University of California, Berkeley, Berkeley, CA, May 12 2011.
- [387] Travis Swicegood. Pragmatic Guide to Git. Pragmatic Bookshelf. The Pragmatic Programmers, Raleigh, NC, November 1 2010.
- [388] Apostolos Syropoulos, Antonis Tsolomitis, and Nick Sofroniou. Digital Typography Using L^AT_EX. Springer Professional Computing. Springer-Verlag New York, New York, NY, 2003.
- [389] Alexander Tarlinder. Developer Testing: Building Quality into Software. Addison-Wesley Signature Series. Addison-Wesley, Boston, MA, 2017.
- [390] Risk Danger technoweenie Olson, Brandon Keepers, Anatoly Borodin, Lucas Werkmeister, Steve Streeting, and Scott Barron. Git large file storage. Available online from *GitHub: GitHub* and *GitHub: Git Large File Storage* at: <https://git-lfs.github.com/> and <https://github.com/github/git-lfs>; April 1, 2016 was the last accessed date, March 31 2016.
- [391] The SciPy community. NumPy reference. Available online from *SciPy.org: Numpy and Scipy Documentation: NumPy v1.17 Manual* as release 1.17 at: <https://docs.scipy.org/doc/numpy/reference/>; self-published; January 20, 2020 was the last accessed date, July 26 2019.
- [392] Eno Thereska. C++ coding standard. Available online from *Carnegie Mellon University: College of Engineering: Department of Electrical and Computer Engineering: Eno Thereska's home page* at: <https://users.ece.cmu.edu/~eno/coding/CppCodingStandard.html>; February 16, 2016 was the last accessed date.
- [393] Shyamkumar Thoziyoor, Jay Brockman, and Daniel Rinzler. Pim lite: a multithreaded processor-in-memory prototype. In Proceedings of the 15th ACM Great Lakes Symposium on VLSI (GLSVLSI '05), pages 64–69, Chicago, IL, April 17–19 2005. Association for Computing Machinery, ACM Press.
- [394] TINYhr Inc. staff. The era of personal and peer accountability: The TINYpulse 2015 employee engagement and organizational culture report. Technical report, TINYhr Inc., Seattle, WA, 2015.
- [395] Adam Tornhill. Software Design X-Rays: Fix Technical Debt with Behavioral Code Analysis. The Pragmatic Programmers, Raleigh, NC, 2018.

- [396] Allen B. Tucker and Robert E. Noonan. Programming Languages: Principles and Paradigms. McGraw-Hill, New York, NY, second edition, 2007.
- [397] Kate L. Turabian, Wayne C. Booth, Gregory G. Colomb, Joseph M. Williams, Joseph Bizup, William T. FitzGerald, and The University of Chicago Press Editorial Staff. A Manual for Writers of Research Papers, Theses, and Dissertations: Chicago Style for Students and Researchers. Chicago Guides to Writing, Editing, and Publishing. The University of Chicago Press, Chicago, IL and London, U.K., ninth edition, 2018.
- [398] Kate L. Turabian, Wayne C. Booth, Gregory G. Colomb, Joseph M. Williams, and The University of Chicago Press Editorial Staff. A Manual for Writers of Research Papers, Theses, and Dissertations: Chicago Style for Students and Researchers. Chicago Guides to Writing, Editing, and Publishing. The University of Chicago Press, Chicago, IL and London, U.K., seventh edition, 2007.
- [399] Nodir Turakulov, Ted Osborne, and Ackermann Yuriy. Markdown style guide. Available online from *GitHub: Google: styleguide: Google documentation guide* at: <https://github.com/google/styleguide/blob/gh-pages/docguide/style.md>; September 17, 2019 was the last accessed date, January 30 2018.
- [400] Greg L. Turnquist. Python Testing Cookbook: Over 70 simple but incredibly effective recipes for taking control of automate testing using powerful Python testing tools. Quick answers to common problems. Packt Publishing, Birmingham, West Midlands, West Midlands, England, U.K., 2011.
- [401] Vijay K. Vaishnavi and William Kuechler Jr. Design Science Research Methods and Patterns: Innovating Information and Communication Technology. Auerbach Publications, Boca Raton, FL, 2008.
- [402] Joseph S. Valacich, Joey F. George, and Jeffrey A. Hoffer. Essentials of Systems Analysis and Design. Pearson Education, Upper Saddle River, NJ, fifth edition, 2012.
- [403] Jonathan W. Valvano. Embedded Microcomputer Systems: Real Time Interfacing. Cengage Learning, Stamford, CT, third edition, 2007.
- [404] Dimitri van Heesch. Doxygen. Available online from *Eindhoven University of Technology: M.C.G.V. Stack: Dimitri van Heesch's Web page* at: <http://www.stack.nl/~dimitri/doxygen/> and <http://doxygen.org/>; November 4, 2016 was the last accessed date, September 11 2016.
- [405] Guido van Rossum, Barry Warsaw, and Nick Coghlan. Style guide for Python code. Available online from *Welcome to Python.org: Python Developer's Guide: Python Enhancement Proposals: Python Enhancement Proposal 0* at: <https://www.python.org/dev/peps/pep-0008/>; October 20, 2015 was the last accessed date, August 1 2013.
- [406] Doug Vucevic and Wayne Yadow. Testing the Data Warehouse Practicum: Assuring Data Content, Data Structures and Quality. Trafford Publishing, Bloomington, IN, 2012.
- [407] Chao Wang. Abstraction Refinement for Large Scale Model Checking. PhD thesis, University of Colorado at Boulder, Boulder, CO, 2004.
- [408] Chao Wang, Gary D. Hachtel, and Fabio Somenzi. Abstraction Refinement for Large Scale Model Checking. Integrated Circuits and Systems. Springer Science+Business Media, LLC, New York, NY, 2006.

- [409] David A. Watt. *Programming Language Design Concepts*. John Wiley & Sons, Chichester, West Sussex, South East England, England, U.K., 2004.
- [410] John L. Weatherwax. Solution manuals and various book notes. Available online from *Dr. John L. Weatherwax's Web page: Continuing Education & Solution Manuals: Publication Categories: Worked Problems, Solution Manuals & Notes from Various Books: Solution Manuals And Various Book Notes* at: http://waxworksmath.com/ce_solutionmanuals.asp; self-published; May 31, 2015 was the last accessed date, 2008.
- [411] Charles W. Wessner. Government-industry partnerships for the development of new technologies. Technical report, National Research Council, Washington, D.C., 2003.
- [412] James A. Whittaker. *Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design*. Addison-Wesley, Boston, MA, 2009.
- [413] Andreas Wieferink, Heinrich Meyr, and Rainer Leupers. *Retargetable Processor System Integration into Multi-Processor System-on-Chip Platforms*. Springer Science+Business Media B.V., Dordrecht, The Netherlands, 2008.
- [414] Karl Wieggers. *Peer Reviews in Software: A Practical Guide*. Addison-Wesley Information Technology Series. Addison-Wesley, Reading, MA, 2002.
- [415] Karl Wieggers and Joy Beatty. *Software Requirements*. Best Practices. Microsoft Press, Redmond, WA, third edition, 2013.
- [416] Wikibooks contributors. Exception handling. In *C++ Programming*, chapter 4. Wikimedia Foundation, San Francisco, CA, February 11 2016.
- [417] Wikibooks contributors. Bibliography management. In *LaTeX*, chapter 5.3. Wikimedia Foundation, San Francisco, CA, January 23 2021.
- [418] Wikipedia contributors. Exception safety. Available online from *Wikipedia, The Free Encyclopedia: CATEGORY* at: https://en.wikipedia.org/wiki/Exception_safety; October 29, 2016 was the last accessed date, September 26 2016.
- [419] Wikipedia contributors. Indent style. Available online from *Wikipedia, The Free Encyclopedia: Source code* at: https://en.wikipedia.org/wiki/Indent_style; November 4, 2016 was the last accessed date, October 30 2016.
- [420] Wikipedia contributors. Natural language programming. Available online from *Wikipedia, The Free Encyclopedia: Computer programming* at: https://en.wikipedia.org/wiki/Natural_language_programming; November 4, 2016 was the last accessed date, November 2 2016.
- [421] Wikipedia contributors. Semipredicate problem. Available online from *Wikipedia, The Free Encyclopedia: Programming language topics* at: https://en.wikipedia.org/wiki/Semipredicate_problem; October 29, 2016 was the last accessed date, October 23 2016.
- [422] Wikipedia contributors. Texinfo. Available online from *Wikipedia, The Free Encyclopedia: Markup languages* at: <https://en.wikipedia.org/wiki/Texinfo>; November 4, 2016 was the last accessed date, April 29 2016.

- [423] Wikipedia contributors. Naming convention (programming). Available online from *Wikipedia, The Free Encyclopedia: Source code* at: [https://en.wikipedia.org/wiki/Naming_convention_\(programming\)](https://en.wikipedia.org/wiki/Naming_convention_(programming)); March 10, 2017 was the last accessed date, February 26 2017.
- [424] Wikipedia contributors. Programming paradigm. Available online from *Wikipedia, The Free Encyclopedia: Programming paradigms* at: https://en.wikipedia.org/wiki/Programming_paradigm; July 12, 2017 was the last accessed date, July 5 2017.
- [425] Wikipedia contributors. Code refactoring. Available online from *Wikipedia, The Free Encyclopedia: 21st-century revolutions* at: https://en.wikipedia.org/wiki/Code_refactoring; September 25, 2018 was the last accessed date, September 20 2018.
- [426] Wikipedia contributors. Compilers: Principles, techniques, and tools. Available online from *Wikipedia, The Free Encyclopedia: Compiler construction* at: https://en.wikipedia.org/wiki/Compilers:_Principles,_Techniques,_and_Tools; January 27, 2023 was the last accessed date, December 26 2021.
- [427] Wikipedia contributors. Introduction to algorithms. Available online from *Wikipedia, The Free Encyclopedia: Computer science books* at: https://en.wikipedia.org/wiki/Introduction_to_Algorithms; January 27, 2023 was the last accessed date, May 9 2022.
- [428] Wikipedia contributors. Citation. Available online from *Wikipedia, The Free Encyclopedia: Bibliography* at: <https://en.wikipedia.org/wiki/Citation>; March 7, 2023 was the last accessed date, February 25 2023.
- [429] Wikipedia contributors. Parenthetical referencing. Available online from *Wikipedia, The Free Encyclopedia: Style guides* at: https://en.wikipedia.org/wiki/Parenthetical_referencing; March 7, 2023 was the last accessed date, January 11 2023.
- [430] Ron Wilson. Agile development in the SoC design world? Available online from *Embedded.com: Insights: Agile Collection* at: <https://www.embedded.com/electronics-blogs/agile-collection/4218869/Agile-development-in-the-SoC-design-world->; September 25, 2018 was the last accessed date, August 17 2011.
- [431] Glynn Winskel. The Formal Semantics of Programming Languages: An Introduction. The MIT Press, Cambridge, MA, 1993.
- [432] Clifford Wolf. Clifford on programming style. Available online from *Clifford Wolf's Personal Homepage* at: <http://www.clifford.at/style.html>; self-published; March 14, 2016 was the last accessed date.
- [433] Marty J. Wolf. The acm code of ethics: a call to action. Communications of the ACM, 59(12):6, December 2016.
- [434] Robert K. Wysocki. Effective Project Management: Traditional, Agile, Extreme. John Wiley & Sons, Indianapolis, IN, seventh edition, 2014.
- [435] Xiong Ying. Changing font style in LaTeX. Available online from *LaTeX-Tutorial.com: Symbols: LaTeX Text Formatting: Font Styles in LaTeX: Changing Font Style in LaTeX* at: <https://latex-tutorial.com/changing-font-style/>; self-published; May 30, 2022 was the last accessed date.

-
- [436] Jeff Younker. Foundations of Agile Python Development. Apress, Berkeley, CA, 2008.
 - [437] Andreas Zeller. Why Programs Fail: A Guide to Systematic Debugging. Morgan Kaufmann, Burlington, MA, second edition, 2009.
 - [438] Kaiping Zeng and Sorin A. Huss. RAMS: A VHDL-AMS code refactoring tool supporting high level analog synthesis. In Proceedings of the 6th IEEE Computer Society Annual Symposium on VLSI: New Frontiers in VLSI Design (ISVLSI'05), Tampa, FL, May 11–12 2005. IEEE Computer Society, IEEE Press.
 - [439] Kaiping Zeng and Sorin A. Huss. Architecture refinements by code refactoring of behavioral VHDL-AMS models. In Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS 2006), pages 1475–1478, Island of Kos, Greece, May 21–24 2006. IEEE Press.
 - [440] Naijun Zhan, Shuling Wang, and Hengjun Zhao. Formal Verification of Simulink/Stateflow Diagrams: A Deductive Approach. Springer International Publishing AG, Cham, Switzerland, 2017.
 - [441] Mark Zuckerberg. Letter from Mark Zuckerberg. In Form S-1 REGISTRATION STATEMENT Under The Securities Act of 1933, Facebook, Inc. U.S. Securities and Exchange Commission, Washington, D.C., February 1 2012.